Final – Evidence-Based Release

Learning Management System

Full-Stack Learning Management Platform (Repository `FSJP_LMS`)

Complete Repository Documentation

Product, Business, Technical, Architecture and Quality Documentation Suite

Product name (UI brand)	Edvanta
Version	1.0
Date	October 7, 2026
Author / Generator	Generated from Repository Analysis
Repository root	FSJP_LMS/
Document status	Final – Evidence-Based Release

Technology stack
React 18 / Vite 6 / Ant Design 5 / Tailwind CSS 3 – Spring Boot 3.3.4 / Java 21 / Spring Security + JWT / Spring Data JPA – MySQL 8 – Maven / Vitest

Confidentiality notice: This document was produced by automated analysis of the repository source. It describes only functionality present in the codebase. Internal engineering and product use.

Generated from Repository Analysis – no author names, metrics, customers or features were invented for this document.

Contents

Document Conventions **x**

1 Project Overview **1**
1.1 What this repository contains . . . 1
1.2 Repository layout . . . 1
1.3 Technology stack . . . 1
1.4 Workstreams and contributors . . . 2
1.5 Version control state . . . 3
1.6 Current maturity . . . 3
1.7 Scope of this documentation suite . . . 3

2 Project Description **4**
2.1 Product identity . . . 4
2.2 Problem statement . . . 4
2.3 Solution . . . 4
2.4 Target audience and personas . . . 5
2.5 Value proposition . . . 5
2.6 Core feature inventory . . . 5
2.7 Product workflow . . . 6
2.8 Use cases . . . 8
2.9 Differentiators (repository evidence only) . . . 8
2.10 Calls to action present in the UI . . . 8
2.11 Visual identity . . . 8
2.12 SEO keywords and AEO/GEO pointers . . . 8

3 Executive Summary **9**
3.1 What the product is . . . 9
3.2 The problem it solves . . . 9
3.3 The solution in one paragraph . . . 9
3.4 Core functionality . . . 9
3.5 Target users . . . 9
3.6 Technology and architecture . . . 10
3.7 Current maturity . . . 11
3.8 Key limitations and risks (headline items) . . . 11
3.9 Bottom line . . . 12

4 Business Requirements Document (BRD) **12**
4.1 Document purpose . . . 12
4.2 Business problem . . . 12
4.3 Business objectives . . . 13
4.4 Stakeholders . . . 13
4.5 User groups . . . 14
4.6 Business processes . . . 14
4.7 Business requirements . . . 14
4.8 Expected outcomes . . . 15

4.9 Success metrics . . . 15
4.10 Constraints . . . 16
4.11 Assumptions . . . 16
4.12 Risks . . . 16

5 Product Requirements Document (PRD) 17
5.1 Product vision . . . 17
5.2 Problem statement . . . 17
5.3 Goals and objectives . . . 17
5.4 Personas . . . 18
5.5 User stories . . . 18
5.6 Use cases . . . 19
5.7 Features and functional requirements . . . 19
5.8 Acceptance criteria . . . 19
5.9 Success metrics . . . 20
5.10 Constraints . . . 20
5.11 Dependencies . . . 20
5.12 Risks . . . 21
5.13 Future scope . . . 21

6 Functional Requirements 21
6.1 Authentication and account management (FR-AUTH) . . . 22
6.2 Course management (FR-COURSE) . . . 23
6.3 Lessons and learning delivery (FR-LESSON) . . . 23
6.4 Enrollment and progress (FR-ENR) . . . 24
6.5 Assessments (FR-QUIZ) . . . 25
6.6 Social features (FR-SOC) and certificates (FR-CERT) . . . 26
6.7 Administration (FR-ADM) and practice (FR-PRA) . . . 27
6.8 Cross-cutting functional behaviour . . . 27

7 Non-Functional Requirements 27
7.1 Security NFRs . . . 28
7.2 Performance NFRs . . . 29
7.3 Scalability NFRs . . . 30
7.4 Usability, accessibility and compatibility . . . 31
7.5 Reliability, maintainability and compliance . . . 32

8 Technical Requirements 33
8.1 Technology baseline . . . 33
8.2 Runtime and hosting requirements . . . 33
8.3 API requirements . . . 34
8.4 Data requirements . . . 34
8.5 Authentication and authorisation requirements . . . 34
8.6 Infrastructure requirements . . . 34
8.7 Integration requirements . . . 35
8.8 Testing requirements . . . 35
8.9 Monitoring and observability requirements . . . 35

9 System Architecture **36**
9.1 Architecture style . . . 36
9.2 Runtime components . . . 38
9.3 Trust boundaries . . . 38
9.4 Deployment view . . . 38
9.5 Architecture decisions (observed) . . . 38

10 Software Architecture **39**
10.1 Module map . . . 39
10.2 Backend packages (software units) . . . 41
10.3 Frontend software units . . . 41
10.4 Component interaction . . . 42
10.5 Data ownership and consistency . . . 44
10.6 Cross-cutting concerns . . . 44

11 High-Level Design **44**
11.1 Design scope . . . 44
11.2 Service inventory and communication . . . 45
11.3 Primary control flows . . . 45
11.4 API surface overview . . . 47
11.5 Data contracts . . . 47
11.6 State management design . . . 48
11.7 Error handling design . . . 48

12 Low-Level Design **48**
12.1 Backend – service layer detail . . . 48
12.2 Backend – representative method detail . . . 50
12.3 Backend – persistence access . . . 50
12.4 Backend – data initialisation logic . . . 50
12.5 Frontend – state and data access detail . . . 51
12.6 Frontend – key component behaviours . . . 51
12.7 Dependencies (internal) . . . 52

13 Frontend Architecture **52**
13.1 Application entry and providers . . . 52
13.2 Routing . . . 52
13.3 Guard semantics . . . 53
13.4 Navigation structure . . . 53
13.5 API layer . . . 54
13.6 Component inventory . . . 54
13.7 Practice module structure . . . 55
13.8 Build pipeline . . . 55

14 Backend Architecture **56**
14.1 Layered structure . . . 56
14.2 Component inventory . . . 56
14.3 Request lifecycle . . . 56
14.4 Security components in detail . . . 57

14.5 DTO and mapping strategy . . . 57
14.6 Exception taxonomy . . . 57
14.7 Configuration and bootstrap . . . 58
14.8 Build and packaging . . . 58

15 Database Design **58**
15.1 Technology and conventions . . . 58
15.2 Entity-relationship diagram . . . 59
15.3 Tables . . . 60
15.3.1 users . . . 60
15.3.2 categories . . . 60
15.3.3 courses . . . 60
15.3.4 lessons . . . 60
15.3.5 enrollments . . . 61
15.3.6 progress . . . 61
15.3.7 quizzes . . . 61
15.3.8 questions . . . 61
15.3.9 quiz_attempts . . . 62
15.3.10 quiz_attempt_answers . . . 62
15.3.11 discussions . . . 62
15.3.12 feedback . . . 62
15.3.13 certificates . . . 62
15.4 Relationship summary . . . 63
15.5 Indexes and constraints . . . 63
15.6 Views, stored procedures, functions, triggers . . . 64
15.7 Row-level security and database access control . . . 64
15.8 Migrations and schema evolution . . . 64
15.9 Entity/schema parity notes . . . 64

16 API Documentation **64**
16.1 Base conventions . . . 64
16.2 Endpoint inventory (47 mapped endpoints, 11 controllers) . . . 65
16.3 Error model . . . 65

17 Authentication **66**
17.1 Mechanism . . . 66
17.2 Validation chain . . . 66
17.3 Limitations . . . 67

18 Authorization **67**
18.1 Model . . . 67
18.2 Authorization matrix . . . 67

19 User Flows **68**
19.1 Covered flows . . . 68
19.2 Flow details . . . 70

20 Data Flows **70**
20.1 Request lifecycle . . . 70
20.2 Key data chains . . . 70
20.3 What does not exist . . . 73

21 UI/UX **73**
21.1 Page inventory . . . 73
21.2 State contract . . . 73
21.3 Responsive and accessible behaviour . . . 73

22 Design System **73**
22.1 Tokens . . . 73
22.2 Component library . . . 73
22.3 Usage rule . . . 74

23 Security **74**
23.1 Implemented controls . . . 74
23.2 Findings . . . 74
23.3 Production requirements . . . 74

24 Performance **75**
24.1 Observed characteristics . . . 75
24.2 Bottleneck risks . . . 75
24.3 Recommendations . . . 75

25 Scalability **75**
25.1 Current posture . . . 75
25.2 Scaling path . . . 76

26 Testing **76**
26.1 Backend tests . . . 76
26.2 Frontend tests . . . 77
26.3 Coverage contract . . . 77

27 Deployment **77**
27.1 Verified development topology . . . 77
27.2 Containerisation . . . 77
27.3 Production checklist . . . 77

28 CI/CD **77**
28.1 Current state . . . 77
28.2 Recommended pipeline . . . 78

29 Integrations **78**
29.1 Embedded practice editor . . . 78
29.2 External origins . . . 78
29.3 Not integrated . . . 78

30 Configuration **78**
30.1 Backend configuration . . . 78
30.2 Frontend configuration . . . 78
30.3 Database bootstrap . . . 78

31 SEO **79**
31.1 Current state . . . 79
31.2 Recommendations . . . 79

32 AEO (Answer-Engine Optimisation) **79**
32.1 Current state . . . 79
32.2 Recommendations . . . 79

33 GEO (Generative-Engine Optimisation) **79**
33.1 Current state . . . 79
33.2 Recommendations . . . 79

34 Known Issues **80**
34.1 Recorded defects and inconsistencies . . . 80

35 Technical Debt **80**
35.1 Debt register . . . 80

36 Roadmap **81**
36.1 Reconciled position . . . 81
36.2 Phased plan . . . 81

37 Development Guidelines **83**
37.1 Layer discipline . . . 83
37.2 Working rules . . . 83

38 Contribution Guide **83**
38.1 Workflow . . . 83
38.2 Definition of Done (abridged) . . . 83

39 Glossary **84**

A References **84**

List of Figures

1 Implemented product workflow – learning lifecycle across the three roles 7
2 System architecture at a glance 11
3 System architecture – runtime components and external dependencies 37
4 Backend software modules and their dependency direction 40
5 Frontend component architecture and data flow to the API 43
6 Authentication and authorisation control flow 46
7 Quiz submission sequence – server-side authoritative scoring 47
8 Entity-relationship diagram of the 13 LMS tables 59
9 Authentication flow – register, login, JWT issuance, per-request validation 66
10 Learning lifecycle – registration through certificate 69
11 Quiz submission sequence – client attempt, server validation, persisted result . . . 70
12 Backend layering that every data flow traverses – controller, service, repository . . 72
13 Deployment view – the only containerised component is the backend JAR 76
14 Certificate eligibility – the gate all roadmap completion work must preserve 82

List of Tables

1 Status classification scheme used throughout this document . . . x
2 Top-level repository structure . . . 1
3 Technology stack as declared in package manifests . . . 2
4 Workstream ownership declared in CONTRACT.md . . . 3
5 Product identity attributes extracted from the repository . . . 4
6 Personas inferred from seeded accounts and role capabilities . . . 5
7 Feature inventory with primary implementation evidence . . . 6
8 Architecture summary for stakeholders . . . 10
9 Headline limitations for leadership attention (full detail in Section 23 and Section 34) 12
10 Declared objectives versus implementation status . . . 13
11 Stakeholder map . . . 14
12 Business requirements register . . . 15
13 Available (engineering) success signals . . . 16
14 Business-facing risks with repository evidence . . . 17
15 PRD goals mapped to implemented capability . . . 18
16 Primary user stories . . . 19
17 External and internal dependencies . . . 20
18 Functional requirements – authentication . . . 22
19 Functional requirements – courses . . . 23
20 Functional requirements – lessons and player . . . 23
21 Functional requirements – enrollment and progress . . . 24
22 Functional requirements – quizzes . . . 25
23 Functional requirements – discussions, feedback, certificates . . . 26
24 Functional requirements – administration and practice module . . . 27
25 Security non-functional requirements . . . 28
26 Performance non-functional requirements . . . 29
27 Scalability non-functional requirements . . . 30
28 Usability and accessibility requirements . . . 31
29 Reliability and maintainability requirements . . . 32
30 Technical baseline and realised versions . . . 33
31 Infrastructure requirement status . . . 35
32 Runtime components and responsibilities . . . 38
33 Software modules and ownership boundaries . . . 41
34 Backend package responsibilities . . . 41
35 Frontend module responsibilities . . . 42
36 Cross-cutting concerns and their implementation points . . . 44
37 Major services and communication channels . . . 45
38 API surface by domain . . . 47
39 Service classes, responsibilities and key methods . . . 49
40 Frontend state, storage and data access . . . 51
41 Representative component logic . . . 51
42 Frontend route table . . . 53
43 Frontend API modules and the endpoints they call . . . 54
44 Component inventory (selected, with purpose) . . . 55
45 Backend component counts . . . 56

46 Security component responsibilities 57
47 Exception types and HTTP mapping (from GlobalExceptionHandler) 58
48 Database technology and design conventions 59
49 Foreign-key relationships and cardinalities 63
50 Controller to endpoint-group mapping (verified in controller sources) 65
51 Authorization matrix (backend-enforced; frontend mirrors for UX) 67
52 Security findings grounded in repository evidence 74
53 Known issues with repository evidence 80
54 Technical debt register 81
55 Project glossary – terms as used in this repository 84

Document Conventions

Every significant claim in this documentation suite carries a status classification:

Table 1: Status classification scheme used throughout this document

Classification	Meaning
IMPLEMENTED	Behaviour verified in repository source code; a file path or symbol is cited as evidence.
INFERRED	A conclusion drawn from code structure or configuration, not stated explicitly anywhere.
RECOMMENDED	A suggestion for improvement; it is *not* implemented and must not be read as a feature.
UNKNOWN	Information that cannot be determined from the repository (for example: runtime metrics, business KPIs).
NOT APPLICABLE	The concern does not exist in this repository (for example: no payment system, no CI/CD pipeline).

Evidence is cited using exact repository paths such as `backend/src/main/java/com/example/lms/service/Cou` or `frontend/src/api/client.js`. Business, product and roadmap statements are separated from engineering facts, and recommendations are never presented as shipped functionality.

1 Project Overview

1.1 What this repository contains

Status: IMPLEMENTED

The repository `FSJP_LMS` is a full-stack **Learning Management System (LMS)** – a web platform that connects learners, instructors and administrators through a centralised online-learning environment. The product UI is branded **Edvanta** (page title "Edvanta – Learn Your Way" and footer "© 2026 Edvanta" in `frontend/index.html` and `frontend/src/components/Footer.jsx`).

The repository implements the complete core learning lifecycle: registration and login, course discovery, enrollment, lesson consumption, progress tracking, quizzes with server-side scoring, discussions, course feedback, and verifiable course-completion certificates. **Status:** IMPLEMENTED

1.2 Repository layout

Status: IMPLEMENTED

Table 2: Top-level repository structure

Path	Contents
`backend/`	Spring Boot 3.3.4 REST API (Java 21), Maven build, Dockerfile, tests
`frontend/`	React 18 SPA (Vite 6), Ant Design 5, Tailwind CSS 3, Vitest tests
`database/`	`schema.sql` (13 tables), `seed.sql`, `dummy.sql` sample data
`README.md`	Project overview, setup instructions, roadmap, authorization matrix
`CONTRACT.md`	Two-workstream engineering contract (ownership, API, DoD)
`.env.example`	Documented backend and frontend environment variables
`.gitignore`	Ignores build output, dependencies, `.env*` files
`audit-dropped.cjs`	Node utility script comparing UI strings against a git baseline
`.vscode/`	Editor launch/settings configuration

Absent from the repository (verified by directory listing): `.github/` (no CI/CD workflows), `docker-compose.yml`, a root `LICENSE` file, and any frontend `Dockerfile`. **Status:** IMPLEMENTED (absence verified)

1.3 Technology stack

Status: IMPLEMENTED

Versions are taken from `backend/pom.xml` and `frontend/package.json`.

Table 3: Technology stack as declared in package manifests

Layer	Technology	Evidence / Purpose
Frontend framework	React 18.3.1	`frontend/package.json` – SPA UI
Build tool	Vite 6.0.1	`frontend/vite.config.js` – dev server on port 3000
UI component library	Ant Design 5.22.2	Tables, modals, forms, tabs, message toasts
Styling	Tailwind CSS 3.4.16	`frontend/tailwind.config.js` – design tokens
Routing	React Router 6.28.0	`frontend/src/routes/AppRoutes.jsx` – 17 routes
HTTP client	Axios 1.7.9	`frontend/src/api/client.js` – interceptors, JWT header
Backend framework	Spring Boot 3.3.4	`backend/pom.xml` parent version
Language	Java 21	`backend/pom.xml` `<java.version>` and Dockerfile
Security	Spring Security + JJWT 0.12.6	Stateless JWT authentication, BCrypt
Persistence	Spring Data JPA / Hibernate	13 repositories, `ddl-auto: update`
Database	MySQL 8 (runtime), H2 (tests)	`database/schema.sql`, `backend/pom.xml`
API documentation	springdoc-openapi 2.6.0	`/v3/api-docs`, `/swagger-ui.html`
Testing	Spring Test, Vitest 2.1.8	`backend/src/test`, `frontend/src/test`
Containerisation	Docker (backend only)	`backend/Dockerfile` multi-stage build

1.4 Workstreams and contributors

Status: IMPLEMENTED

`CONTRACT.md` defines a two-workstream split of ownership:

Table 4: Workstream ownership declared in CONTRACT.md

Workstream	Primary	Backup	Owns
A – Frontend	Jay	Ali	`frontend/**` (UI, routing, state, tests)
B – Backend	Ali	Jay	`backend/**`, `database/**` (API, auth, schema, tests)
Shared	Jay + Ali	–	`README.md`, `CONTRACT.md`, `.env.example`, API contract, integration

The names *Jay* and *Ali* appear only as workstream labels taken verbatim from `CONTRACT.md` and the OpenAPI contact block in `backend/src/main/java/com/example/lms/config/OpenApiConfig.java`; no other authorship information exists in the repository.

1.5 Version control state

Status: IMPLEMENTED

At the time of analysis the repository had **15 commits** on the checked-out branch `frontend/practice-module`. Local branches: `main`, `frontend/foundation-implementation`, `frontend/practice-module`. Remote branches across three remotes (`origin`, `lms-frontend`, `upstream`) follow the pattern `backend/foundation-implementation`, `frontend/foundation-implementation`, `frontend/practice-modul` Commit messages follow the conventional style mandated by the contract (for example `feat(frontend): add coding practice pages and routes`). **Status:** IMPLEMENTED

1.6 Current maturity

Status: INFERRED

Evidence supporting the maturity assessment:

- A full vertical slice of the learning lifecycle exists and is covered by an end-to-end backend test (`backend/src/test/java/com/example/lms/integration/LmsIntegrationTest.java` – register, course, lessons, quiz, enroll, progress, certificate, discussion, feedback).
- The frontend integrates the real API through `frontend/src/api/` (no mock API layer except the explicitly isolated practice dataset).
- 22 backend test methods (10 classes) and 39 frontend test methods (8 files) exist.
- There is no CI/CD pipeline, no deployment configuration beyond a single backend Dockerfile, no licence file, and no production environment documentation.

Overall maturity: **feature-complete core MVP in active development; not production-hardened**. This is an assessment, not a repository claim.

1.7 Scope of this documentation suite

Status: IMPLEMENTED

This document was produced by reading the repository source, configuration, SQL, tests and git history. Sections are modular files under `documents/latex/sections/`. Where the repository contains no evidence for a topic (business metrics, service-level objectives, CI/CD, payments), the section records that absence rather than inventing content.

2 Project Description

This section is written for product managers, designers, developers, marketing teams and AI-assisted landing-page generators. Every product claim is grounded in the repository; anything inferred is labelled as such.

2.1 Product identity

Status: IMPLEMENTED

Table 5: Product identity attributes extracted from the repository

Attribute	Value (evidence)
Working title	Learning Management System (`README.md`, `CONTRACT.md`)
UI brand name	Edvanta (`frontend/index.html` title, `frontend/src/components/Footer.jsx`)
Tagline (literal)	"Learn Your Way" (`frontend/index.html <title>`)
Hero headline	"Empower your future with structured learning and verified credentials." (`frontend/src/pages/HomePage.jsx`)
Meta description	"LMS Portal – structured courses, rigorous assessments and verified certificates." (`frontend/index.html`)
Product category	B2C/B2B Education Technology – Learning Management System
Delivery model	Single-page web application (responsive, browser-based)
Licence claim	"MIT License" stated in `README.md`; **no LICENSE file exists**

2.2 Problem statement

Status: INFERRED

Derived from the solution implemented in code: online learning content, assessments, progress records and credentials are typically scattered across tools, making it difficult to run a complete learn→assess→certify cycle in one place. The repository exists to provide one platform where that full lifecycle is executed end-to-end (the lifecycle diagram is part of `README.md` and `CONTRACT.md`).

2.3 Solution

Status: IMPLEMENTED

A role-based web platform where:

- **Learners** discover courses, enrol, watch/read lessons, track progress, take quizzes, discuss, review, and earn printable certificates.
- **Instructors** author courses, order lessons, create quizzes with questions and answer keys, publish content, and review learner submissions.
- **Administrators** manage users, course categories and the course catalogue, and monitor platform data.

All authority (scoring, eligibility, ownership) is enforced server-side in the Spring Boot backend – see Section 18.

2.4 Target audience and personas

Status: IMPLEMENTED (roles) / INFERRED (personas)

The three user roles are implemented in code (`User.Role` enum in `backend/src/main/java/com/example/lms`: `LEARNER`, `INSTRUCTOR`, `ADMIN`). Persona narratives below are *inferred* from the seeded demo accounts in `backend/src/main/java/com/example/lms/config/DataInitializer.java` and `database/seed.sq` no real customer research exists in the repository.

Table 6: Personas inferred from seeded accounts and role capabilities

Persona	Seeded account	Goals supported by implemented features
Learner (Alex Learner)	`learner@example.com`, role LEARNER	Browse/enrol, complete lessons, pass quizzes, track progress, hold certificates, join discussions, review courses
Instructor (Sarah Instructor)	`instructor@example.com`, role INSTRUCTOR	Create and publish courses, manage lessons, author quizzes/questions, view learner submissions and scores
Administrator (Admin User)	`admin@example.com`, role ADMIN	Manage users and categories, delete any course, view all quiz attempts, administer the platform

2.5 Value proposition

Status: IMPLEMENTED

1. **Complete lifecycle in one system** – from registration to a downloadable certificate, verified by the end-to-end integration test.
2. **Authoritative assessment** – quiz scores are computed by the backend against stored answer keys; the client never determines pass/fail (`backend/src/main/java/com/example/lms/service/QuizServi`
3. **Verifiable credentials** – each certificate carries a unique code (`LMS-XXXXXXXX-NNNN`) and is only issued when 100% of lessons are complete and every quiz is passed (`backend/src/main/java/com/example`
4. **Instructor self-service authoring** – courses, ordered lessons, quizzes and question banks are managed through the Instructor Studio UI without developer involvement.
5. **Built-in coding practice** – course-attached coding problems with an embedded OneCompiler editor (Section 29).

2.6 Core feature inventory

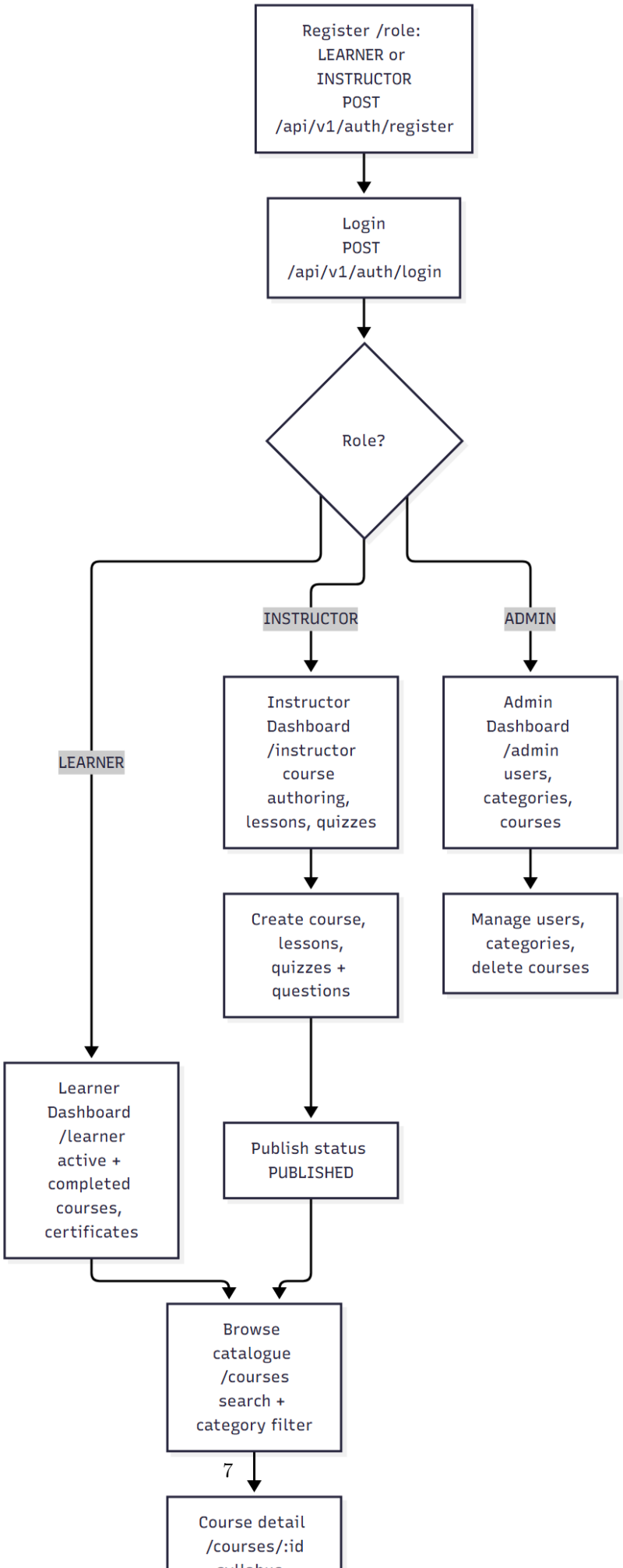
Status: IMPLEMENTED

Table 7: Feature inventory with primary implementation evidence

Feature	Primary evidence	Status
Registration / login / logout (JWT)	backend/src/main/java/com/example/lms/controller/AuthController.ja frontend/src/context/AuthContext.jsx	IMPLEMENTED
Role-based dashboards	frontend/src/components/ Learner, Instructor and Admin dashboards	IMPLEMENTED
Course catalogue, search, category filter	backend/src/main/java/com/example/lms/controller/CourseController frontend/src/pages/CourseCatalogPage.jsx	IMPLEMENTED
Course CRUD + publish states	backend/src/main/java/com/example/lms/service/CourseService.java, InstructorDashboard	IMPLEMENTED
Lesson management + player	backend/src/main/java/com/example/lms/controller/LessonController frontend/src/components/LessonPlayer.jsx	IMPLEMENTED
Enrollment / unenrolment (soft drop)	backend/src/main/java/com/example/lms/service/EnrollmentService.ja	IMPLEMENTED
Progress tracking per lesson and course	backend/src/main/java/com/example/lms/service/ProgressService.java	IMPLEMENTED
Quizzes, question banks, timed attempts	backend/src/main/java/com/example/lms/service/QuizService.java, frontend/src/components/QuizPage.jsx	IMPLEMENTED
Discussions with threaded replies	backend/src/main/java/com/example/lms/service/DiscussionService.ja DiscussionList/DiscussionItem	IMPLEMENTED
Course feedback (1–5 rating, one per user)	backend/src/main/java/com/example/lms/service/FeedbackService.java FeedbackForm	IMPLEMENTED
Certificates with eligibility rules	backend/src/main/java/com/example/lms/service/CertificateService. CertificateCard	IMPLEMENTED
Coding practice workspace (OneCompiler)	frontend/src/components/practice/	IMPLEMENTED (local dataset)
Category administration	backend/src/main/java/com/example/lms/controller/CategoryControlle AdminDashboard	IMPLEMENTED
OpenAPI/Swagger documentation	backend/src/main/java/com/example/lms/config/OpenApiConfig.java	IMPLEMENTED

2.7 Product workflow

Status: IMPLEMENTED


Register /role: LEARNER or INSTRUCTOR POST /api/v1/auth/register
Login POST /api/v1/auth/login
Role?
LEARNER
INSTRUCTOR
ADMIN
Instructor Dashboard /instructor course authoring, lessons, quizzes
Admin Dashboard /admin users, categories, courses
Create course, lessons, quizzes + questions
Manage users, categories, delete courses
Learner Dashboard /learner active + completed courses, certificates
Publish status PUBLISHED
Browse catalogue /courses search + category filter
Course detail /courses/:id

2.8 Use cases

Status: IMPLEMENTED

1. **Self-paced learner**: registers, browses the catalogue, enrols, completes lessons in order, passes a timed quiz, and prints a certificate from the dashboard.
2. **Instructor content author**: creates a course in draft, adds ordered lessons with video links or reading text, authors a quiz with answer keys, publishes, then reviews learner submissions and scores.
3. **Platform administrator**: seeds and curates categories, reviews the user directory, removes courses, and audits all quiz attempts.
4. **Practice learner**: opens `/practice` or a course's practice tab and solves coding problems inside the embedded OneCompiler editor against a static problem set.

2.9 Differentiators (repository evidence only)

Status: IMPLEMENTED

- **Enforced certificate eligibility**: all lessons completed *and* every quiz passed – implemented server-side, tested in `CertificateServiceTest.java`.
- **Course-scoped coding practice**: static, versioned practice problems attached to real course IDs, with starter-code injection via `postMessage` into the OneCompiler iframe.
- **Engineering contract as product governance**: `CONTRACT.md` fixes ownership, API change control, and a Definition of Done – unusual discipline for a project of this size.

Not differentiators, honestly stated: there is no recommendation engine, no payments, no mobile app, no video hosting, and no analytics – all explicitly excluded in `CONTRACT.md` section 5.2.

2.10 Calls to action present in the UI

Status: IMPLEMENTED

Home page (`frontend/src/pages/HomePage.jsx`): "Browse all courses" (primary), "Join for free" (logged-out) or "Go to dashboard" (logged-in), closing dark CTA band. Auth pages: "Log in to Edvanta", "Create Your Account". Course pages: "Enroll Now" / "Continue Learning" (`frontend/src/components/EnrollmentButton.jsx`). Additional CTA ideas are **Status:** RECOMMENDED and intentionally not listed as product copy.

2.11 Visual identity

Status: IMPLEMENTED

Brand gradient `linear-gradient(90deg, #fc4c02 0%, #ef2cc1 52%, #bdbbff 100%)` rendered by `frontend/src/components/ui/GradientRibbon.jsx`; black-first UI (`colorPrimary: #000000`), mint accent `#c8f6f9`, dark canvas `#010120`. Full token tables are in Section 22.

2.12 SEO keywords and AEO/GEO pointers

Status: INFERRED (keywords) / IMPLEMENTED (metadata)

Literal metadata supports the keywords "LMS", "structured courses", "assessments", "certificates", "learn". **Status:** RECOMMENDED additional keyword families ("online learning platform", "course management system") are *not* present in the repository and must be added deliberately – see Sections 31, 32 and 33 for the gap analysis.

3 Executive Summary

This summary is written for non-technical stakeholders. All statements are traceable to the repository.

3.1 What the product is

Status: IMPLEMENTED

Edvanta (Learning Management System) is a browser-based platform for delivering online courses with assessments, progress tracking and completion certificates. It serves three roles – learners, instructors and administrators – through a single responsive web application backed by a secure REST API.

3.2 The problem it solves

Status: INFERRED

Running a complete learning programme requires stitching together content delivery, testing, progress records and credential issuance. This product implements that entire cycle in one system, from account registration through to a printable, uniquely coded certificate.

3.3 The solution in one paragraph

Status: IMPLEMENTED

Learners enrol in courses and work through ordered lessons; the platform records lesson-level progress and derives course completion percentages. Instructors author courses, lessons and multiple-choice quizzes; when a learner submits a quiz the *server* scores it against stored answer keys and persists the attempt. When a learner has completed every lesson of a course and passed every quiz in it, the backend issues a certificate carrying a unique code, the learner's name, the course title and an issue date. Administrators curate categories, review the user directory and moderate the catalogue.

3.4 Core functionality

Status: IMPLEMENTED

- JWT authentication with BCrypt password hashing
- Role-based access control (learner / instructor / admin)
- Course catalogue with search and category filters
- Course, lesson and quiz authoring (instructor)
- Enrollment with duplicate protection
- Lesson player with video/embed and text content
- Per-lesson and per-course progress tracking
- Timed quizzes, server-side scoring, answer explanations
- Threaded course discussions
- Course feedback with 1–5 star ratings
- Certificate generation with eligibility enforcement
- Coding-practice workspace with embedded OneCompiler editor
- Admin console for users, categories and courses
- Live OpenAPI/Swagger API documentation

3.5 Target users

Status: IMPLEMENTED

Three account roles are implemented in the database schema (`role` column, default `LEARNER`; `database/schema.sql`) and enforced by Spring Security annotations (Section 18). Typical users: self-paced learners, content-creating instructors, and platform administrators.

3.6 Technology and architecture

Status: IMPLEMENTED

Table 8: Architecture summary for stakeholders

Concern	Choice
Client	React 18 single-page application built with Vite; Ant Design components with Tailwind styling
Server	Spring Boot 3.3.4 (Java 21) layered REST API: controller → service → repository
Data store	MySQL 8 with 13 related tables managed through JPA/Hibernate
Security	Stateless JSON Web Tokens (24-hour expiry), BCrypt hashing, method-level role checks
API style	Versioned JSON REST (`/api/v1/...`) wrapped in a consistent `{success, message, data}` envelope
Quality tooling	22 backend tests (JUnit/Mockito/MockMvc), 39 frontend tests (Vitest + Testing Library)

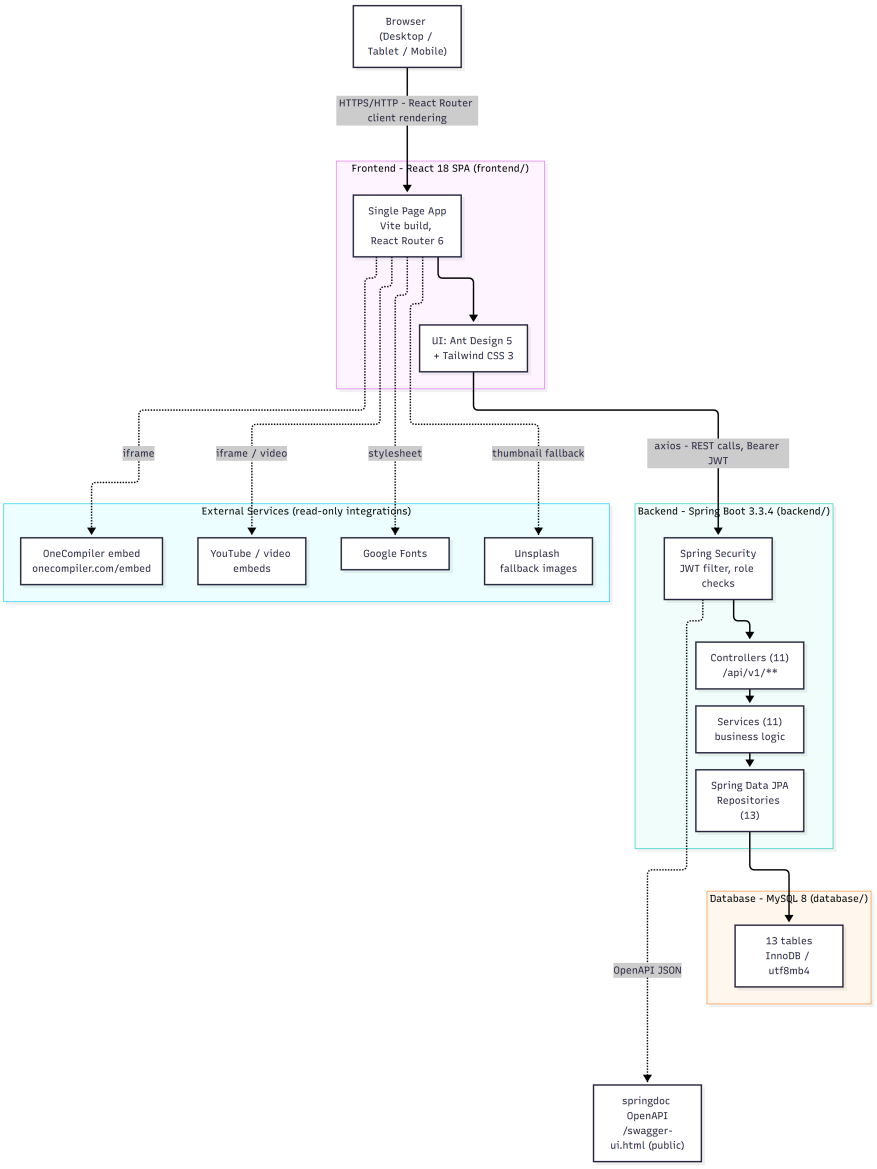


Figure 2: System architecture at a glance

3.7 Current maturity

Status: INFERRED

- **Working:** the end-to-end learning lifecycle passes an automated integration test; the frontend consumes the real API (no mock server); 61 automated tests exist across both tiers.
- **Not yet in place:** CI/CD, production deployment configuration, database migration tooling, licence file, rate limiting, monitoring/alerting, accessibility audit, SEO infrastructure (sitemap/structured data).
- **Repository signals:** 15 commits, active feature branches, contract-driven two-person workflow.

3.8 Key limitations and risks (headline items)

Status: IMPLEMENTED (verified findings)

Table 9: Headline limitations for leadership attention (full detail in Section 23 and Section 34)

Severity	Finding (evidence)
Critical	Open registration honours a caller-supplied `role` field – an anonymous request can create an ADMIN account (`backend/src/main/java/com/example/lms/service/AuthService.java`)
Critical	Default admin credentials seeded on every fresh database (`admin@example.com`/`admin123`; `backend/src/main/java/com/example/lms/config/DataInitializer.java`)
High	Default JWT signing secret is committed to source; tokens are forgeable if the environment variable is unset
High	CORS allows any origin together with credentials (`SecurityConfig.java`)
Medium	Logout is client-side only – the token stays valid for 24 hours
Medium	No pagination on any list endpoint; dashboard performs sequential per-enrolment requests
Medium	Several UI defects: footer links to non-existent routes, dashboard deep links do not switch tabs
Low	No CI/CD, no lint configuration, unused dependency (`canvas-confetti`)

3.9 Bottom line

Status: INFERRED

The repository delivers a credible, tested MVP of the full learn–assess–certify cycle with clear engineering governance (`CONTRACT.md`). Before any production exposure, the critical and high security findings above must be remediated and a deployment/CI pipeline must be established. These are recommendations, not current capabilities.

4 Business Requirements Document (BRD)

4.1 Document purpose

This BRD records the business-level requirements that can be derived from the repository: `README.md` (objectives, roles), `CONTRACT.md` (scope, exclusions, Definition of Done) and the implemented behaviour of the code. No revenue model, customer list, market data or KPI targets exist anywhere in the repository; such items are marked **Status:** UNKNOWN.

4.2 Business problem

Status: INFERRED (from README objectives and implemented scope)

Educational content delivery, learner assessment, progress record-keeping and credential issuance are fragmented across tools. The repository exists to provide a single platform covering the complete cycle:

`Registration` → `Discovery` → `Enrollment` → `Learning` → `Assessment` → `Evaluation` → `Completion` → `Certificate`

This lifecycle is stated verbatim in `README.md` and `CONTRACT.md` section 1 and implemented in code.

4.3 Business objectives

Status: IMPLEMENTED (objectives as declared in README.md; implementation status verified below)

Table 10: Declared objectives versus implementation status

Objective (README)	Status	Evidence
Centralised platform for online learning	IMPLEMENTED	Single SPA + API + database
User registration and enrolment	IMPLEMENTED	AuthController, EnrollmentService
Structured course content and materials	IMPLEMENTED	Lesson entity (URL + rich text)
Learner progress tracking	IMPLEMENTED	ProgressService, progress table
Online assessments and quizzes	IMPLEMENTED	QuizService, quiz tables
Performance recording and display	IMPLEMENTED	QuizAttempt, QuizResult UI
Communication around courses	IMPLEMENTED	DiscussionService, DiscussionList
Instructor content management tools	IMPLEMENTED	InstructorDashboard
Centralised system administration	IMPLEMENTED	AdminDashboard, UserController
Certificates after completion	IMPLEMENTED	CertificateService (eligibility enforced)
Secure authentication and RBAC	IMPLEMENTED	Spring Security + `@PreAuthorize`

4.4 Stakeholders

Status: IMPLEMENTED (roles) / INFERRED (interest)

Table 11: Stakeholder map

Stakeholder	Role in project	Interest
Learner (end user)	Platform user	Complete courses, pass assessments, hold certificates
Instructor (content owner)	Platform user	Author and publish content, monitor submissions
Administrator	Platform operator	Curate catalogue, manage users, enforce policy
Frontend workstream (Jay)	Code owner per `CONTRACT.md`	UI/UX quality, frontend tests, integration
Backend workstream (Ali)	Code owner per `CONTRACT.md`	API contract, security, schema, backend tests
AI coding agents	Assisted contributors	Must obey `CONTRACT.md` section 26 rules

4.5 User groups

Status: IMPLEMENTED

The three user groups map directly to the `users.role` column (`database/schema.sql`) and to route-level restrictions in `frontend/src/routes/AppRoutes.jsx` plus server-side rules in `backend/src/main/java/com/example/lms/config/SecurityConfig.java`. Their permissions are tabulated in Section 18.

4.6 Business processes

Status: IMPLEMENTED

1. **Learner acquisition:** register → verify email-uniqueness → receive JWT → browse → enrol.
2. **Content publishing:** instructor creates course (DRAFT) → adds ordered lessons → authors quiz + questions → sets status PUBLISHED → course becomes discoverable.
3. **Learning delivery:** enrol → open classroom → consume lesson (video/embed/text) → mark complete → progress recomputed.
4. **Assessment:** open quiz → answer under time limit → submit → server scores → result persisted and displayed with explanations.
5. **Credentialing:** all lessons complete + all quizzes passed → backend issues certificate → learner prints/saves from dashboard.
6. **Quality loop:** enrolled learners post discussions and one feedback rating per course; instructors view attempt results.

4.7 Business requirements

Status: IMPLEMENTED unless stated otherwise

Table 12: Business requirements register

ID	Requirement	Status
BR-01	The platform must serve learner, instructor and administrator roles in one system	IMPLEMENTED
BR-02	Learners must be able to self-register and immediately receive an authenticated session	IMPLEMENTED
BR-03	Only authorised users may create or modify courses and lessons	IMPLEMENTED
BR-04	Assessment results must be computed by the server, never trusted from the client	IMPLEMENTED
BR-05	Certificates must only be issued when completion requirements are met	IMPLEMENTED
BR-06	The system must prevent duplicate active enrolments and duplicate feedback	IMPLEMENTED
BR-07	The platform must be usable on desktop, tablet and mobile browsers	IMPLEMENTED (responsive classes; no formal audit)
BR-08	The product must be documented for engineering hand-over	IMPLEMENTED (this document)
BR-09	Payment, subscriptions and live classes are out of scope for v1.0	IMPLEMENTED (excluded in `CONTRACT.md` 5.2)
BR-10	Business revenue, market-share or growth targets	UNKNOWN (not present in repository)

4.8 Expected outcomes

Status: IMPLEMENTED (product outcomes) / UNKNOWN (business outcomes)

- A learner can go from registration to a printable certificate without human intervention – proven by `backend/src/test/java/com/example/lms/integration/LmsIntegrationTest.java`.
- An instructor can publish a course with lessons and a quiz entirely through the UI.
- An administrator can curate categories and review platform users and courses.
- **Business** outcomes (enrolment volume, completion rates, revenue) are **Status:** UNKNOWN – the repository contains no analytics, telemetry or reporting pipeline.

4.9 Success metrics

Status: UNKNOWN

No instrumentation for business metrics exists in the repository (no analytics package in `frontend/package.json`, no metrics/actuator dependency in `backend/pom.xml`, no reporting tables in `database/schema.sql`). The only measurable quality signals available today are:

Table 13: Available (engineering) success signals

Signal	Value	Status
Backend automated tests	22 test methods across 10 classes	IMPLEMENTED
Frontend automated tests	39 test methods across 8 files	IMPLEMENTED
End-to-end lifecycle test	1 full-stack integration scenario	IMPLEMENTED
Coverage measurement	none configured (no JaCoCo, no coverage script)	NOT PRESENT

4.10 Constraints

Status: IMPLEMENTED (declared in repository documents)

- Two-person workstream ownership with strict directory boundaries (`CONTRACT.md` sections 3 and 27).
- Fixed technology baseline; major replacements require both contributors' agreement (`CONTRACT.md` section 2).
- Feature work must not alter APIs or schema without coordinated change management (sections 29–30).
- Frontend route guards are explicitly *not* security boundaries – enforcement is a backend duty (section 13).
- No licence file exists despite an MIT claim in `README.md` (legal constraint gap).

4.11 Assumptions

Status: INFERRED

1. MySQL is available to developers at `localhost:3306` with credentials supplied via environment (per `.env.example`).
2. The application is single-tenant (no multi-tenant requirement appears anywhere).
3. English is the only locale (no i18n files exist).
4. HTTPS termination, if any, is handled outside the repository (no TLS configuration exists).

4.12 Risks

Status: IMPLEMENTED (verified conditions)

Table 14: Business-facing risks with repository evidence

Severity	Risk
Critical	Self-registration can mint ADMIN accounts (`role` honoured in `backend/src/main/java/com/example/lms/service/AuthService.java`) – undermines the entire role model
Critical	Default admin/instructor/learner credentials are seeded on empty databases in any environment
High	Committed default JWT secret allows token forgery when `JWT_SECRET` is unset
High	Wildcard CORS with credentials exposes authenticated APIs to any origin
Medium	No CI/CD means regressions are only caught by developers remembering to run tests
Medium	No analytics means business success cannot be measured
Low	MIT licence claimed but LICENSE file missing – redistribution terms unclear

5 Product Requirements Document (PRD)

5.1 Product vision

Status: INFERRED (synthesised from README.md and CONTRACT.md)

Give every learner, instructor and administrator one secure, self-service platform where the full learning lifecycle – discover, enrol, learn, prove, certify – works end-to-end without operational overhead.

5.2 Problem statement

Status: INFERRED

Learners need a trustworthy path from course discovery to a verifiable credential; instructors need authoring and monitoring tools; administrators need catalogue and user control. Today these are separate activities. The product solves them as one integrated flow.

5.3 Goals and objectives

Status: IMPLEMENTED (goals declared in repository) / INFERRED (framing)

Table 15: PRD goals mapped to implemented capability

ID	Goal	Implementation evidence
G1	Secure, role-aware access for three personas	`backend/src/main/java/com/example/lms` + ProtectedRoute
G2	Full course catalogue with search/filter	CourseController + CourseCatalogPage
G3	Structured learning with progress visibility	LessonPlayer, ProgressBar, ProgressService
G4	Authoritative assessment	QuizService scoring, QuizResult review UI
G5	Evidence of completion	CertificateService eligibility + CertificateCard
G6	Engagement: discussion and feedback	DiscussionService, FeedbackService
G7	Self-service authoring for instructors	InstructorDashboard (course/lesson/quiz modals)
G8	Extensible practice layer	`frontend/src/components/practice/` + local dataset
G9	Contract-driven, tested delivery	`CONTRACT.md`, 61 automated tests

5.4 Personas

Covered in Section 2 (personas inferred from seeded accounts; **Status:** INFERRED).

5.5 User stories

Status: IMPLEMENTED (stories exercised by existing code and tests)

Table 16: Primary user stories

ID	Story	Covered by
US-01	As a visitor I can register and log in so that I can use the platform	AuthControllerTest
US-02	As a learner I can browse and search the catalogue without logging in	public GET `/api/v1/courses`
US-03	As a learner I can enrol in a course exactly once	EnrollmentServiceTest
US-04	As a learner I can mark lessons complete and see course progress	ProgressServiceTest
US-05	As a learner I can take a quiz under a time limit and see my score	QuizServiceTest, quiz.test.jsx
US-06	As a learner I can generate a certificate when I complete a course	CertificateServiceTest
US-07	As a learner I can discuss a course and leave one rating/feedback	social_cert.test.jsx
US-08	As an instructor I can create, edit, publish and delete my courses	CourseServiceTest, InstructorDashboard
US-09	As an instructor I can author quizzes with questions and answer keys	InstructorDashboard quiz modal
US-10	As an instructor I can view learner submissions for my quizzes	QuizService results scoping
US-11	As an admin I can manage categories and view all users	CategoryController, UserController
US-12	As any user I can update my profile (name, bio, avatar)	ProfilePage + UserService

5.6 Use cases

Covered in Section 2 (use cases UC-1 to UC-4).

5.7 Features and functional requirements

Status: IMPLEMENTED

The authoritative feature list with acceptance-relevant detail is maintained in Section 6. Feature inventory for product readers is in Section 2.

5.8 Acceptance criteria

Status: IMPLEMENTED (Definition of Done as written in CONTRACT.md section 32)

The repository's own acceptance gate is the Definition of Done. Summarised (full checklist in `CONTRACT.md`):

- **Authentication:** register/login/logout work; passwords hashed; JWT works; protected routes and role authorisation work; invalid authentication handled.
- **Courses:** CRUD works; ownership/permissions enforced.
- **Enrollment:** learner can enrol; duplicates prevented; unauthorised actions rejected.
- **Learning:** lessons load; completion recorded; progress persisted and reflected in the UI.
- **Assessments:** quiz loads; answers submitted; *backend* calculates authoritative score; results persisted and visible.

- **Discussions/Feedback:** creation, retrieval, replies, validation and unauthorised-modification rejection work.
- **Certificates:** requirements validated; correct learner/course data; downloadable.
- **Frontend:** all pages implemented; loading/empty/error states exist; forms validate; responsive layout; tests pass; production build succeeds.
- **Integration:** frontend connects to the real backend; no required feature depends on temporary mock data.

Gap versus gate: the practice module ships with static data by explicit design (`frontend/src/data/practiceDa` header comment), and `CONTRACT.md` allows mocks only during isolated frontend development.
Status: KNOWN DEVIATION

5.9 Success metrics

Status: UNKNOWN

No product analytics exist (see Section 4). **Status:** RECOMMENDED metrics to instrument before launch: registration→enrolment conversion, course completion rate, quiz pass rate, certificate issue rate – none of these can currently be measured.

5.10 Constraints

Status: IMPLEMENTED

- v1.0 scope boundaries (excluded: payments, streaming, mobile apps, AI features, SSO, multi-tenancy) – `CONTRACT.md` section 5.2.
- Two-workstream ownership; cross-boundary changes require coordination (sections 27, 29–31).
- Single language (English), single database, single deployment target implied by configuration.

5.11 Dependencies

Status: IMPLEMENTED

Table 17: External and internal dependencies

Dependency	Use	Status
MySQL 8	Primary data store	Required at runtime
Node.js + npm	Frontend build/test	Required at build time
Java 21 + Maven	Backend build (Maven wrapper 3.9.9 included)	Required at build time
Google Fonts (Inter, JetBrains Mono)	Typography (`frontend/index.html`)	Runtime CDN call
OneCompiler embed	Coding-practice editor iframe	Runtime third-party
YouTube embed URLs	Lesson video players	Content-supplied
Unsplash images	Fallback thumbnails (hardcoded URLs)	Runtime third-party

5.12 Risks

Covered in Section 4 (risks) and Section 34 (defects).

5.13 Future scope

Status: IMPLEMENTED (as declared) / NOT IMPLEMENTED (as content)

`README.md` publishes a four-phase roadmap: Phase 1 (core LMS) is marked complete; Phases 2–4 (discussions/feedback/notifications/analytics, instructor platform, advanced platform incl. password recovery, CI/CD, monitoring) are unchecked. Note that discussions and feedback *are* already implemented while listed as Phase 2 – the README roadmap has drifted from the code. The reconciled roadmap is Section 36.

6 Functional Requirements

Each requirement is traceable to implemented code. **Status:** IMPLEMENTED means behaviour was verified in source; **Status:** RECOMMENDED marks capabilities that do not exist and are suggested, not promised.

6.1 Authentication and account management (FR-AUTH)

Table 18: Functional requirements – authentication

ID	Requirement	Evidence
FR-AUTH-01	The system shall allow self-registration with name, email, password and role	`POST /api/v1/auth/register`
FR-AUTH-02	Passwords shall be stored only as BCrypt hashes	`BCryptPasswordEncoder` in SecurityConfig
FR-AUTH-03	Email uniqueness must be enforced at registration (409 on duplicate)	AuthService `existsByEmail`
FR-AUTH-04	Password policy: minimum 6, maximum 100 characters	`@Size` on RegisterRequest
FR-AUTH-05	Successful authentication returns a signed JWT (24 h) plus the user profile	AuthResponse
FR-AUTH-06	Login failures return 401 `UNAUTHORIZED` with generic message	AuthService + GlobalExceptionHandler
FR-AUTH-07	Subsequent requests authenticate via `Authorization: Bearer` header	JwtAuthenticationFilter
FR-AUTH-08	Profile update (name, bio, avatar URL; optional password) for self or admin	UserService.updateUser
FR-AUTH-09	*Client* logout clears stored token and calls `POST /auth/logout`	AuthContext.logout
FR-AUTH-10	Server-side logout must invalidate the token	**Status:** RECOMMENDED (not implemented – endpoint is a no-op)

6.2 Course management (FR-COURSE)

Table 19: Functional requirements – courses

ID	Requirement	Evidence
FR-CRS-01	List courses with optional `search`, `categoryId`, `status`, `instructorId` filters	CourseController GET
FR-CRS-02	Retrieve a single course incl. computed `totalLessons`, `totalEnrolled`	CourseService.mapToCourseResponse
FR-CRS-03	Create a course restricted to INSTRUCTOR/ADMIN (403 otherwise)	`@PreAuthorize` on POST
FR-CRS-04	Update/delete restricted to admin or the owning instructor	`validateCourseOwnership`
FR-CRS-05	Course status lifecycle: DRAFT, PUBLISHED, ARCHIVED	`Course.Status` enum
FR-CRS-06	Course fields: title, description, category, thumbnail URL, instructor	CourseCreateRequest
FR-CRS-07	Category CRUD restricted to ADMIN; duplicate names rejected (409)	CategoryController/Service
FR-CRS-08	Public (unauthenticated) read access to course list and detail	SecurityConfig permitAll GET

6.3 Lessons and learning delivery (FR-LESSON)

Table 20: Functional requirements – lessons and player

ID	Requirement	Evidence
FR-LSN-01	Lessons belong to a course with explicit `orderIndex` and `durationMinutes`	Lesson entity
FR-LSN-02	Lesson content: external/video URL, embeddable media, or long-form text	LessonPlayer media detection
FR-LSN-03	Lesson CRUD restricted to instructor-owner/admin	LessonController `@PreAuthorize`
FR-LSN-04	New lessons default to the next order index	LessonService.createLesson
FR-LSN-05	Learner can mark a lesson complete (100%) with watch-time seconds	ProgressController POST/PATCH
FR-LSN-06	Classroom shows curriculum sidebar with completion state and lesson navigation	LearningPage
FR-LSN-07	Video URLs (YouTube/Vimeo/`.mp4`) are converted to embeddable players	`getEmbedUrl` in LessonPlayer

6.4 Enrollment and progress (FR-ENR)

Table 21: Functional requirements – enrollment and progress

ID	Requirement	Evidence
FR-ENR-01	Learner enrols in a course; duplicate active enrolment returns 409	EnrollmentService.enroll
FR-ENR-02	Unenrolment is a soft drop (status DROPPED), rows are retained	EnrollmentService.unenroll
FR-ENR-03	Only self or ADMIN may list a user's enrolments (403 otherwise)	EnrollmentService.getUserEnrollments
FR-ENR-04	Instructors are blocked from enrolling as learners in the UI	EnrollmentButton guard
FR-PRG-01	Progress is stored per user+lesson with unique constraint	`uk_user_lesson_progress`
FR-PRG-02	Course progress % is computed as completed/total lessons (2 dp)	ProgressService.getCourseProgress
FR-PRG-03	Completing all lessons auto-marks the enrolment COMPLETED	`checkAndUpdateCourseCompletion`
FR-PRG-04	Dashboard distinguishes active vs completed enrolments	LearnerDashboard

6.5 Assessments (FR-QUIZ)

Table 22: Functional requirements – quizzes

ID	Requirement	Evidence
FR-QIZ-01	Quizzes belong to a course with passing score (default 70%) and time limit (default 30 min)	Quiz entity/service defaults
FR-QIZ-02	Questions have four options, a correct option (A–D), points (default 10) and an explanation	Question entity
FR-QIZ-03	Learners receive questions *without* correct answers; instructors/admins with them	`includeAnswers` flag in QuizService
FR-QIZ-04	Quiz authoring (create/update/delete) restricted to instructor-owner/admin	`@PreAuthorize` + ownership check
FR-QIZ-05	Submission is scored server-side: `earned/possible x 100`, pass if $\geq$ passing score	QuizService.submitQuiz
FR-QIZ-06	Each attempt persists score, total points, passed flag and per-answer correctness	quiz_attempt tables
FR-QIZ-07	Result view shows per-question explanations and correct answers	QuizResult component
FR-QIZ-08	Instructors see attempts for their own quizzes; admins see all	`getQuizResults` scoping
FR-QIZ-09	Empty quizzes cannot be submitted (400)	`no questions to evaluate` check
FR-QIZ-10	Scoring denominator shall include unanswered questions	**Status:** RECOMMENDED (current code counts only submitted answers)

6.6 Social features (FR-SOC) and certificates (FR-CERT)

Table 23: Functional requirements – discussions, feedback, certificates

ID	Requirement	Evidence
FR-SOC-01	Threaded course discussions (top-level posts + nested replies, depth-limited in UI)	Discussion entity self-reference
FR-SOC-02	Only the author or ADMIN may delete a discussion (403 otherwise)	DiscussionService.deleteDiscussion
FR-SOC-03	Feedback requires enrolment; one rating (1–5) per user/course; updates allowed	FeedbackService checks
FR-SOC-04	Course review page shows average rating and per-star distribution	FeedbackForm
FR-CERT-01	Certificate generation requires enrolment	CertificateService eligibility check
FR-CERT-02	All lessons of the course must be completed (when lessons exist)	`completedLessons < totalLessons` guard
FR-CERT-03	Every quiz in the course must have a passed attempt	`existsByQuizIdAndUserIdAndPassedTru`
FR-CERT-04	Unique certificate code (`LMS-XXXXXXXX-NNNN`) + issue date + learner/course data	CertificateService
FR-CERT-05	Generation is idempotent – existing certificate is returned	`orphan` check first
FR-CERT-06	Learner can print/save the certificate as PDF from the browser	CertificateCard `window.print()`
FR-CERT-07	Public certificate verification endpoint	**Status:** RECOMMENDED (README Phase 4 item; not implemented)

6.7 Administration (FR-ADM) and practice (FR-PRA)

Table 24: Functional requirements – administration and practice module

ID	Requirement	Evidence
FR-ADM-01	List all users (ADMIN only, enforced server-side)	`GET /api/v1/users` `@PreAuthorize`
FR-ADM-02	Admin console with KPI tiles, user directory, category CRUD, course moderation	AdminDashboard
FR-ADM-03	Role changes only performed by ADMIN	UserService.updateUser
FR-ADM-04	Bootstrap data: 4 default categories and 3 demo accounts on empty DB	DataInitializer
FR-PRA-01	Global practice workspace listing 7 problems (3 languages) with difficulty badges	PracticePage + practiceData.js
FR-PRA-02	Course-scoped practice tab for problems attached to a course	CoursePracticeTab
FR-PRA-03	Embedded OneCompiler editor with starter-code injection via `postMessage`	OneCompilerEditor
FR-PRA-04	Practice data served from a backend API	**Status:** RECOMMENDED (explicitly local today; single-file swap designed in)

6.8 Cross-cutting functional behaviour

- **Consistent envelopes:** every success body is `{success, message, data}`; every error body is `{error: {code, message, details?}}` (`backend/src/main/java/com/example/lms/dto/response/ApiErrorResponse.java`).
- **Validation:** Bean Validation (`@Valid`) on all request DTOs; field errors aggregated into `VALIDATION_ERROR` with per-field details.
- **UI states:** every major page implements loading / success / empty / error / retry states as mandated by `CONTRACT.md` section 16 (LoadingState, ErrorState, EmptyState components).
- **Public reads:** unauthenticated users can browse courses and categories; everything else requires a token (SecurityConfig).

7 Non-Functional Requirements

Non-functional requirements (NFRs) are assessed against the repository. **Status:** IMPLEMENTED = satisfied by code; **Status:** PARTIAL = partially satisfied with evidence; **Status:** NOT IMPLEMENTED = no evidence; **Status:** UNKNOWN = cannot be determined from the repository.

7.1 Security NFRs

Table 25: Security non-functional requirements

ID	Requirement	Assessment
NFR-SEC-01	Passwords stored hashed (never plaintext)	IMPLEMENTED – BCrypt
NFR-SEC-02	Authentication via short-lived signed tokens	PARTIAL – 24 h JWT, no rotation/revocation
NFR-SEC-03	Authorisation enforced server-side on every protected operation	PARTIAL – method security present; several service-level gaps (Section 23)
NFR-SEC-04	Secrets supplied via environment, never committed	NOT IMPLEMENTED – default JWT secret and seed credentials committed
NFR-SEC-05	Safe CORS configuration	NOT IMPLEMENTED – wildcard origin + credentials
NFR-SEC-06	SQL injection prevention	IMPLEMENTED – JPA parameterised queries, no string-concatenated SQL found
NFR-SEC-07	XSS prevention	IMPLEMENTED (client) – React escapes output; no `dangerouslySetInnerHTML` found
NFR-SEC-08	CSRF protection	BY DESIGN N/A – stateless JWT, CSRF disabled; acceptable only with strict CORS (currently violated)
NFR-SEC-09	Rate limiting / brute-force protection	NOT IMPLEMENTED – no rate limiter dependency or filter
NFR-SEC-10	Transport security (HTTPS) enforcement	NOT IMPLEMENTED in repository
NFR-SEC-11	Audit logging of privileged actions	NOT IMPLEMENTED – no audit table/appender

7.2 Performance NFRs

Table 26: Performance non-functional requirements

ID	Requirement	Assessment
NFR-PERF-01	API response targets (e.g. p95 < 500 ms)	UNKNOWN – no benchmarks, thresholds or APM exist
NFR-PERF-02	List endpoints shall paginate large datasets	NOT IMPLEMENTED – all list endpoints return full collections
NFR-PERF-03	HTTP client shall use bounded timeouts	IMPLEMENTED – axios timeout 15 000 ms
NFR-PERF-04	Repeated lookups should be cached	NOT IMPLEMENTED – no cache abstraction; every request hits the DB
NFR-PERF-05	Avoid chatty client patterns	PARTIAL – known N+1 waterfall in LearnerDashboard; per-keystroke catalogue search
NFR-PERF-06	Database indexes for hot access paths	IMPLEMENTED – 20 declared indexes in `database/schema.sql`
NFR-PERF-07	Payload efficiency	PARTIAL – no field projection; full entity graphs serialised through DTOs

7.3 Scalability NFRs

Table 27: Scalability non-functional requirements

ID	Requirement	Assessment
NFR-SCL-01	Server shall be stateless to allow horizontal scaling	IMPLEMENTED – `SessionCreationPolicy.STATELESS`, JWT in client
NFR-SCL-02	Multiple backend instances shall run behind a load balancer	NOT IMPLEMENTED – no deployment topology in repository
NFR-SCL-03	Schema evolution shall use versioned migrations	NOT IMPLEMENTED – `ddl-auto: update` only; no Flyway/Liquibase
NFR-SCL-04	Read scalability via replicas/connection pooling	UNKNOWN – relies on driver defaults; no pool tuning
NFR-SCL-05	Frontend shall serve static assets via CDN	NOT IMPLEMENTED – plain Vite `dist/` output

7.4 Usability, accessibility and compatibility

Table 28: Usability and accessibility requirements

ID	Requirement	Assessment
NFR-USA-01	Responsive usability on desktop, tablet, mobile	IMPLEMENTED (contract section 17; responsive Tailwind classes, mobile drawer nav)
NFR-USA-02	Loading, empty, error and retry states on major pages	IMPLEMENTED – LoadingState/ErrorState/EmptyState + tests
NFR-USA-03	WCAG 2.1 AA conformance	PARTIAL – `role="alert"`, `role="status"`, `role="progressbar"`, focus-visible styles exist; no audit, no axe tests, missing landmarks/alt text coverage
NFR-USA-04	Consistent form validation messages	IMPLEMENTED – antd rules + server `VALIDATION_ERROR` details
NFR-USA-05	Browser support statement	UNKNOWN – none declared; Vite default modern-browser targets

7.5 Reliability, maintainability and compliance

Table 29: Reliability and maintainability requirements

ID	Requirement	Assessment
NFR-REL-01	Consistent error handling across the API	IMPLEMENTED – GlobalExceptionHandler, 6 exception types
NFR-REL-02	Automated regression suite run in CI	PARTIAL – suites exist (61 tests) but no CI executes them
NFR-REL-03	Structured logging with sensible levels	PARTIAL – SLF4J levels configured; no JSON/appender/rotation strategy
NFR-REL-04	Health checks / readiness probes	NOT IMPLEMENTED – no Actuator dependency
NFR-REL-05	Backup and recovery procedure	NOT IMPLEMENTED – no scripts or docs
NFR-MAINT-01	Layered, documented architecture	IMPLEMENTED – layered backend, modular frontend, CONTRACT.md
NFR-MAINT-02	Naming conventions enforced	IMPLEMENTED – canonical naming lists in `CONTRACT.md` section 15
NFR-MAINT-03	Linting/static analysis	NOT IMPLEMENTED – no ESLint/Prettier/SpotBugs configuration
NFR-CMP-01	Personal-data handling (privacy policy, consent)	NOT IMPLEMENTED – no privacy policy artefacts
NFR-CMP-02	Licence clarity	NOT IMPLEMENTED – MIT claimed in README; no LICENSE file

8 Technical Requirements

This document records the technical requirements *as realised* in the repository. Where a requirement is conventional for this class of system but absent here, it is recorded as such.

8.1 Technology baseline

Status: IMPLEMENTED

`CONTRACT.md` section 2 fixes the baseline; `backend/pom.xml` and `frontend/package.json` pin the actual versions:

Table 30: Technical baseline and realised versions

Area	Realised version	Notes
Java	21	`<java.version>21</java.version>`; README prerequisites say "Java 17+" (drift)
Spring Boot	3.3.4	Parent POM; starter-web, data-jpa, security, validation
JJWT	0.12.6	HMAC-SHA signing, `parseSignedClaims` API
springdoc-openapi	2.6.0	OpenAPI 3 + Swagger UI
React	18.3.1	Function components, hooks, StrictMode
Vite	6.0.1	Dev server port 3000, `/api` proxy to 8080
Ant Design	5.22.2	Custom theme tokens in `frontend/src/App.jsx`
Tailwind CSS	3.4.16	Extended token theme in `frontend/tailwind.config.js`
React Router	6.28.0	Declarative route tree with guards
Axios	1.7.9	Instance with request/response interceptors
Vitest	2.1.8	jsdom environment, Testing Library
MySQL	8.x (schema targets utf8mb4/InnoDB)	H2 2.x in-memory for tests (`MODE=MySQL`)
Maven	Wrapper 3.9.9	`backend/.mvn/wrapper/maven-wrapper.properties`
Docker base images	eclipse-temurin 21 (alpine)	Multi-stage backend image

8.2 Runtime and hosting requirements

- **Backend runtime:** Java 21 JRE, port `PORT` (default 8080), JDBC connectivity to MySQL (environment `.env.example`).
- **Frontend runtime:** static assets served by any HTTP server after `npm run build`; during development served by Vite on port 3000.
- **Database:** MySQL 8 with schema from `database/schema.sql`; Hibernate `ddl-auto: update` also manages schema at boot.
- **No orchestration requirements are defined** (no docker-compose, Kubernetes manifests,

or process manager configs exist).
Status: IMPLEMENTED

8.3 API requirements

- All endpoints under `/api/v1`; JSON request/response; consistent envelopes (Section 16).
- Authentication: Bearer JWT in `Authorization` header; token TTL configured by `JWT_EXPIRATION` (default 86 400 000 ms).
- Authorisation: URL-level rules in `SecurityConfig` plus method-level `@PreAuthorize` plus ownership checks inside services.
- Error contract: HTTP status + `{error: {code, message, details?}}` body; stack traces never exposed (catch-all returns a generic message).
- API documentation: machine-readable `/v3/api-docs` and human Swagger UI `/swagger-ui.html`, both public.
- Open change-management rules for breaking changes (`CONTRACT.md` section 29).

Status: IMPLEMENTED

8.4 Data requirements

- 13 entities / tables with foreign keys and unique constraints (Section 15).
- Identifier strategy: database-generated `BIGINT` identity columns (not UUIDs).
- Timestamps maintained by JPA lifecycle callbacks (`@PrePersist`/`@PreUpdate`).
- Referential integrity via `ON DELETE CASCADE` (child rows) and `SET NULL` (category on course delete).
- Seed/bootstrap data: `database/seed.sql`, `database/dummy.sql` (SQL) plus runtime bootstrap in `DataInitializer.java`.

Status: IMPLEMENTED

8.5 Authentication and authorisation requirements

- Stateless JWT sessions; BCrypt password hashing; roles `LEARNER`, `INSTRUCTOR`, `ADMIN` persisted as strings.
- Frontend guards (`frontend/src/routes/ProtectedRoute.jsx`) are usability controls only; the backend is the security boundary.
- Token payload claims: subject = user id, `email`, `role`, issued-at, expires-at.

Status: IMPLEMENTED (with gaps listed in Section 23)

8.6 Infrastructure requirements

Status: PARTIAL

Table 31: Infrastructure requirement status

Requirement	Status	Evidence
Reproducible backend container build	IMPLEMENTED	`backend/Dockerfile`
Full-stack local environment definition	NOT IMPLEMENTED	no docker-compose.yml
CI/CD pipeline	NOT IMPLEMENTED	no `.github/`
Secrets management	PARTIAL	env vars documented; defaults committed
Static hosting for frontend	NOT IMPLEMENTED	no hosting config
Monitoring/alerting	NOT IMPLEMENTED	no Actuator/Prometheus/Alerting
Log aggregation	NOT IMPLEMENTED	console logging only
Database backup automation	NOT IMPLEMENTED	absent

8.7 Integration requirements

- OneCompiler embedded editor (iframe + `postMessage` starter-code injection) – Section 29.
- Video providers (YouTube/Vimeo/`.mp4`) handled client-side by URL rewriting in `frontend/src/components/l`
- Google Fonts (Inter, JetBrains Mono) loaded at page level; Unsplash fallback thumbnails referenced by absolute URL.
- No payment, email, SMS, OAuth/SSO or webhook integrations exist (excluded by contract).

Status: IMPLEMENTED

8.8 Testing requirements

- Backend: JUnit 5 + Mockito + MockMvc; test profile `backend/src/main/resources/application-test.yml` with H2 (`create-drop`, 1 h JWT).
- Frontend: Vitest + Testing Library with jsdom; setup file `frontend/src/test/setup.js`; mocks per API module.
- Contract requirement: tests are part of feature completion (`CONTRACT.md` sections 18 and 39).
- Missing: coverage reporting, E2E browser tests, performance tests, security tests (all **Status:** NOT IMPLEMENTED).

Status: IMPLEMENTED (suites) / NOT IMPLEMENTED (tooling gaps)

8.9 Monitoring and observability requirements

Status: NOT IMPLEMENTED

Logging levels are configured in `backend/src/main/resources/application.yml` (`com.example.lms: DEBUG` in dev). No metrics, tracing, health endpoints, error-tracking (e.g. Sentry) or log shipping exist in the repository. **Status:** RECOMMENDED before production.

9 System Architecture

9.1 Architecture style

Status: IMPLEMENTED

The system is a classical **three-tier client–server architecture**:

1. **Presentation tier** – React single-page application, rendered client-side, communicating exclusively over JSON REST.
2. **Application tier** – Spring Boot stateless REST API implementing authentication, authorisation and business rules.
3. **Data tier** – MySQL relational store accessed through JPA/Hibernate.

There is no message broker, cache server, search engine, file store or background job runner in the repository: every interaction is a synchronous HTTP request/response cycle.

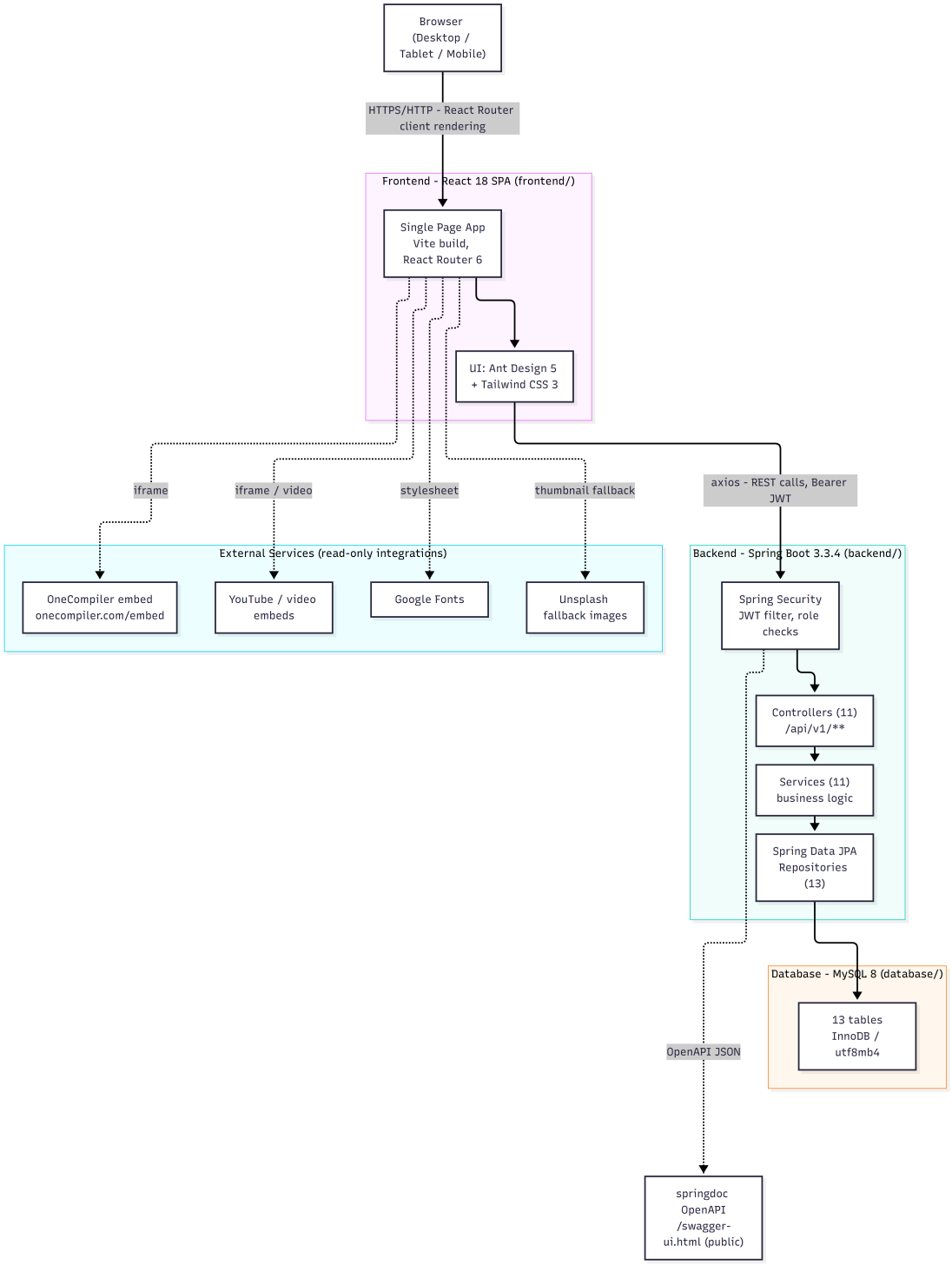


Figure 3: System architecture – runtime components and external dependencies

9.2 Runtime components

Table 32: Runtime components and responsibilities

Component	Responsibility	Key evidence
Browser	Renders SPA, holds session in `localStorage`	`frontend/src/context/AuthContext.jsx`
Vite dev server (dev)	Serves module graph on :3000, proxies `/api`	`frontend/vite.config.js`
Static host (prod)	Serves built `dist/` bundle	`npm run build` script
Axios client	Adds Bearer token, unwraps envelopes, maps errors	`frontend/src/api/client.js`
Spring Security filter chain	JWT validation, role enforcement, CORS	`backend/src/main/java/com/example/lms/cor`
REST controllers (11)	HTTP binding + validation	`backend/src/main/java/com/example/lms/cor`
Services (11)	Business rules, transactions	`backend/src/main/java/com/example/lms/ser`
JPA repositories (13)	Persistence	`backend/src/main/java/com/example/lms/rep`
MySQL	Durable storage, constraints, indexes	`database/schema.sql`
springdoc	OpenAPI JSON + Swagger UI	`backend/src/main/java/com/example/lms/cor`

9.3 Trust boundaries

Status: IMPLEMENTED (design) / PARTIAL (enforcement)

1. **Browser ↔ API:** the only trusted client boundary. Tokens are issued and validated here; everything crossing this boundary is untrusted input validated by Bean Validation.
2. **API ↔ Database:** credentials via environment (`DB_URL` etc.); application connects as configured datasource user.
3. **Client-side state:** `localStorage` tokens are convenience state, not a security boundary – contract section 13 explicitly states frontend guards are not security controls.
4. **Third-party origins:** OneCompiler iframe, YouTube, Google Fonts, Unsplash are outside the trust boundary and receive no user secrets.

9.4 Deployment view

Status: PARTIAL

The only artefact is `backend/Dockerfile` (multi-stage: JDK 21 alpine build with Maven, then JRE 21 alpine runtime, `EXPOSE 8080`). The frontend has no Dockerfile; no compose file ties components together; no production topology is defined. The verified development topology is: MySQL on `localhost:3306` + `./mvnw spring-boot:run` on :8080 + Vite on :3000. See Section 27.

9.5 Architecture decisions (observed)

Status: INFERRED (decisions are implicit in code)

- **Stateless JWT over server sessions** – enables stateless scaling but forfeits revocation (see Section 17).

- **Monolith over microservices** – one deployable JAR; appropriate for current scale; contract forbids unagreed technology changes.
- **Server-authoritative scoring** – correctness and certificates decided in the backend; client is display-only.
- **Hibernate `ddl-auto: update`** over versioned migrations – fast iteration, but schema history is not tracked (risk recorded in Section 35).
- **Static practice dataset** – practice features shipped without backend work by explicit, documented design.

10 Software Architecture

10.1 Module map

Status: IMPLEMENTED

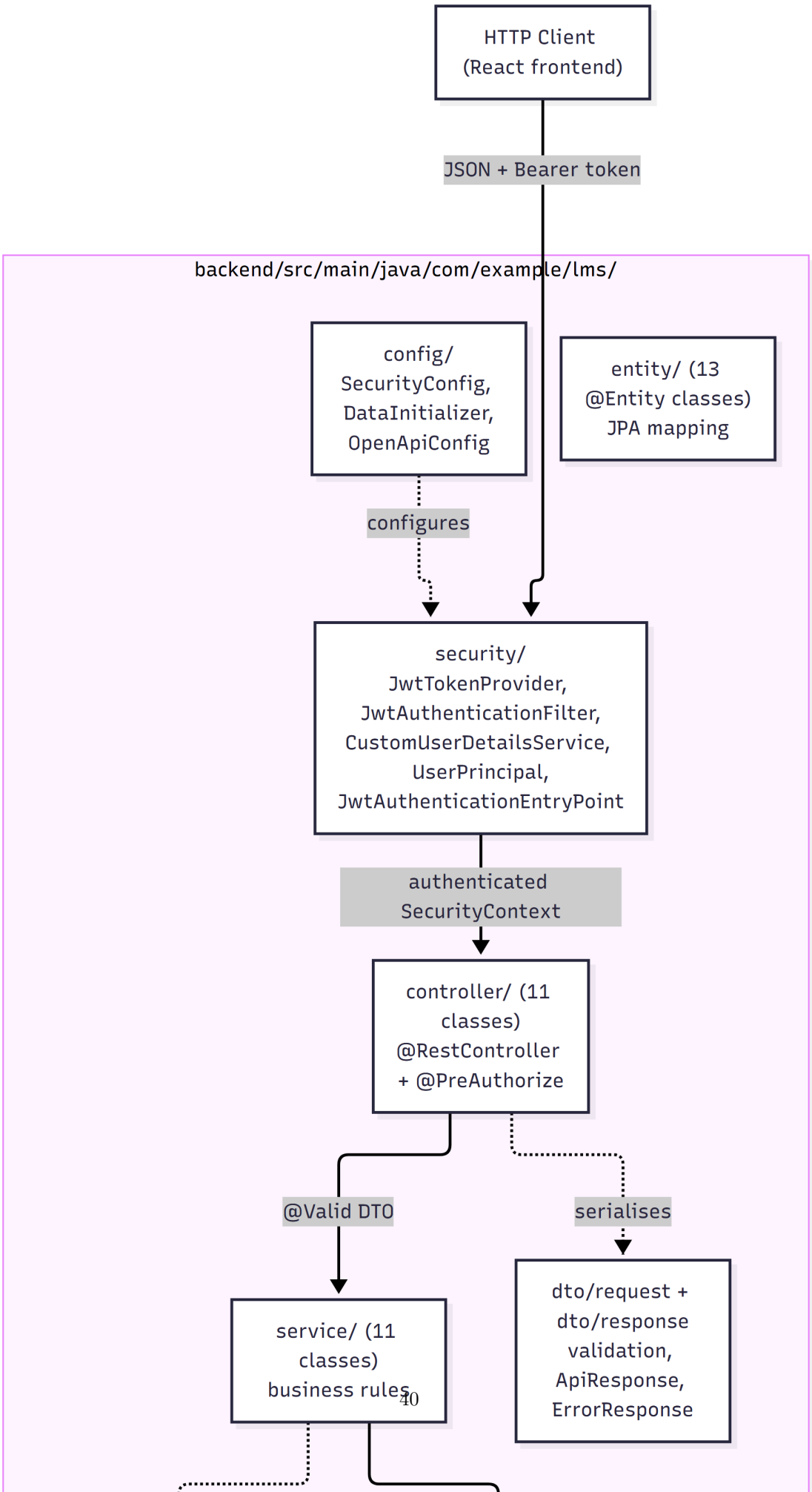

HTTP Client
(React frontend)
JSON + Bearer token
backend/src/main/java/com/example/lms/
config/
SecurityConfig,
DataInitializer,
OpenApiConfig
entity/ (13
@Entity classes)
JPA mapping
configures
security/
JwtTokenProvider,
JwtAuthenticationFilter,
CustomUserDetailsService,
UserPrincipal,
JwtAuthenticationEntryPoint
authenticated
SecurityContext
controller/ (11
classes)
@RestController
+ @PreAuthorize
@Valid DTO
serialises
service/ (11
classes)
business rules
dto/request +
dto/response
validation,
ApiResponse,
ErrorResponse

Table 33: Software modules and ownership boundaries

Module	Contents	Boundary rule
Frontend application	`frontend/src/` (api, components, pages, routes, context, hooks, data, utils)	Must not modify `backend/**` or `database/**`
Backend application	`backend/src/main/java/com/example/lms/` (8 packages)	Must not modify `frontend/**`
Database	`database/*.sql`	Owned exclusively by the backend workstream
API contract	DTO classes + `CONTRACT.md` sections 10–12	Changes require cross-workstream agreement
Test suites	`backend/src/test/`, `frontend/src/test/`	Feature is incomplete until both suites pass

10.2 Backend packages (software units)

Status: IMPLEMENTED

Table 34: Backend package responsibilities

Package	Responsibility	Key classes
`config`	Framework wiring	SecurityConfig, DataInitializer, OpenApiConfig
`security`	Authentication pipeline	JwtTokenProvider, JwtAuthenticationFilter, CustomUserDetailsService, UserPrincipal, JwtAuthenticationEntryPoint
`controller`	HTTP adapters (11)	AuthController, CourseController, QuizController…
`dto`	API contracts (22 classes)	request/* + response/*
`service`	Business logic (11)	CourseService, QuizService, CertificateService…
`repository`	Persistence (13 interfaces)	Spring Data CRUD + derived queries
`entity`	Domain model (13)	User, Course, Lesson, Quiz, Certificate…
`exception`	Error taxonomy	GlobalExceptionHandler + 6 exceptions

10.3 Frontend software units

Status: IMPLEMENTED

Table 35: Frontend module responsibilities

Module	Responsibility	Files
Entry	Bootstrap, antd theme, providers	`src/main.jsx`, `src/App.jsx`
Shell	Navbar, role sidebar/drawer, footer	AppShell, Navbar, Sidebar, Footer
Routing	Route tree + guards	`src/routes/` (AppRoutes, ProtectedRoute)
Pages	Screen-level composition (11 + 2 practice)	`src/pages/`
Components	Reusable UI (40+)	`src/components/` incl. practice/ and ui/
API layer	11 feature modules + shared client	`src/api/`
Auth state	Session context, role derivation	`src/context/AuthContext.jsx`
Practice data	Static problem dataset (swap boundary)	`src/data/practiceData.js`
Utilities	Constants, formatters	`src/utils/`
Tests	Vitest suites (8 files)	`src/test/`

10.4 Component interaction

Status: IMPLEMENTED

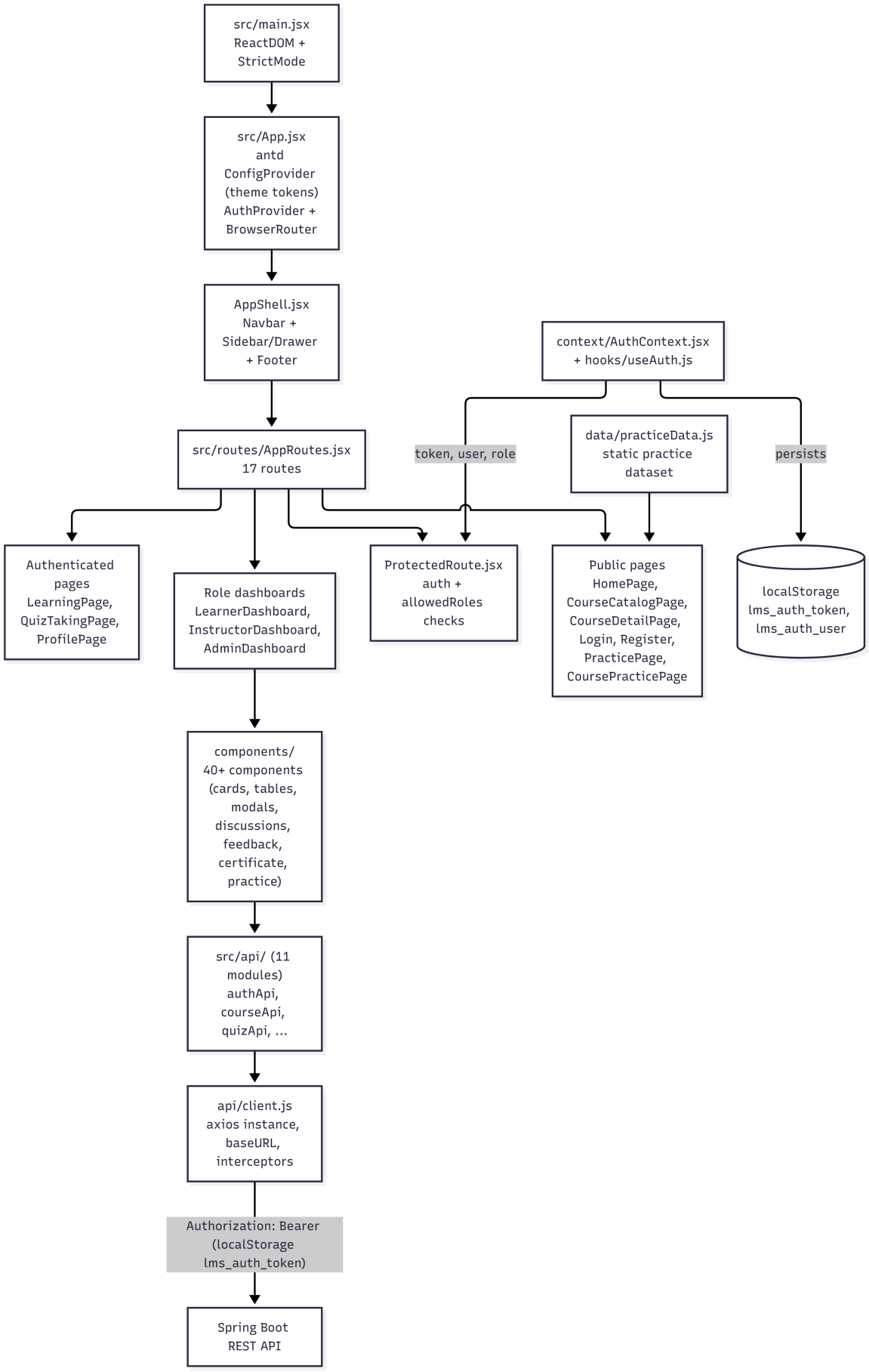


Figure 5: Frontend component architecture and data flow to the API

10.5 Data ownership and consistency

Status: IMPLEMENTED

- MySQL is the single source of truth for users, catalogue, enrolments, progress, attempts, discussions, feedback and certificates.
- The backend owns all writes; the client may only send intentions (e.g. `{completed: true}`), never authoritative results (scores, eligibility).
- Transactions: service methods are `@Transactional` where multi-write (e.g. `AuthService.register`); each HTTP request is a single logical unit of work.
- Optimistic concurrency control (version columns) is **Status:** NOT IMPLEMENTED – concurrent updates last-write-wins.

10.6 Cross-cutting concerns

Table 36: Cross-cutting concerns and their implementation points

Concern	Mechanism	Location
Authentication	JWT filter before `UsernamePasswordAuthenticationFilter`	`security/JwtAuthenticationFilter.java`
Authorisation	URL rules + `@PreAuthorize` + service ownership checks	SecurityConfig, controllers, services
Validation	Bean Validation on DTOs	`dto/request/*`
Error handling	`@RestControllerAdvice` mapping to envelopes	`exception/GlobalExceptionHandler.java`
Logging	SLF4J with package log levels	`application.yml`
API docs	springdoc annotation-driven OpenAPI	`config/OpenApiConfig.java`
Client errors	Axios interceptor maps backend error codes	`frontend/src/api/client.js`
UI states	Loading/Error/Empty primitives	`frontend/src/components/common/`

11 High-Level Design

11.1 Design scope

Status: IMPLEMENTED

High-level design covers the major runtime services and how they communicate. The system has exactly three internal services (frontend, backend, database) communicating synchronously over HTTP/JDBC – there is no asynchronous messaging, no event bus and no scheduled processing in the repository.

11.2 Service inventory and communication

Table 37: Major services and communication channels

Service	Communication	Contract
Frontend → Backend	HTTPS/HTTP JSON over REST (Axios)	`/api/v1/...`, Bearer JWT, `{success, message, data}` envelope
Backend → Frontend	JSON responses / error envelopes	`{error: {code, message, details?}}`
Backend → Database	JDBC via JPA/Hibernate	13 tables, FK constraints, indexed lookups
Frontend → OneCompiler	`iframe` + `postMessage`	embed URL `https://onecompiler.com/embed/...`; message `eventType: populateCode`
Frontend → Media/CDN	`iframe`/`<video>`/`<img>`	YouTube embed URLs, Unsplash images, Google Fonts
Client → OpenAPI	GET `/v3/api-docs`, `/swagger-ui.html`	OpenAPI 3 (publicly readable)

11.3 Primary control flows

Status: IMPLEMENTED

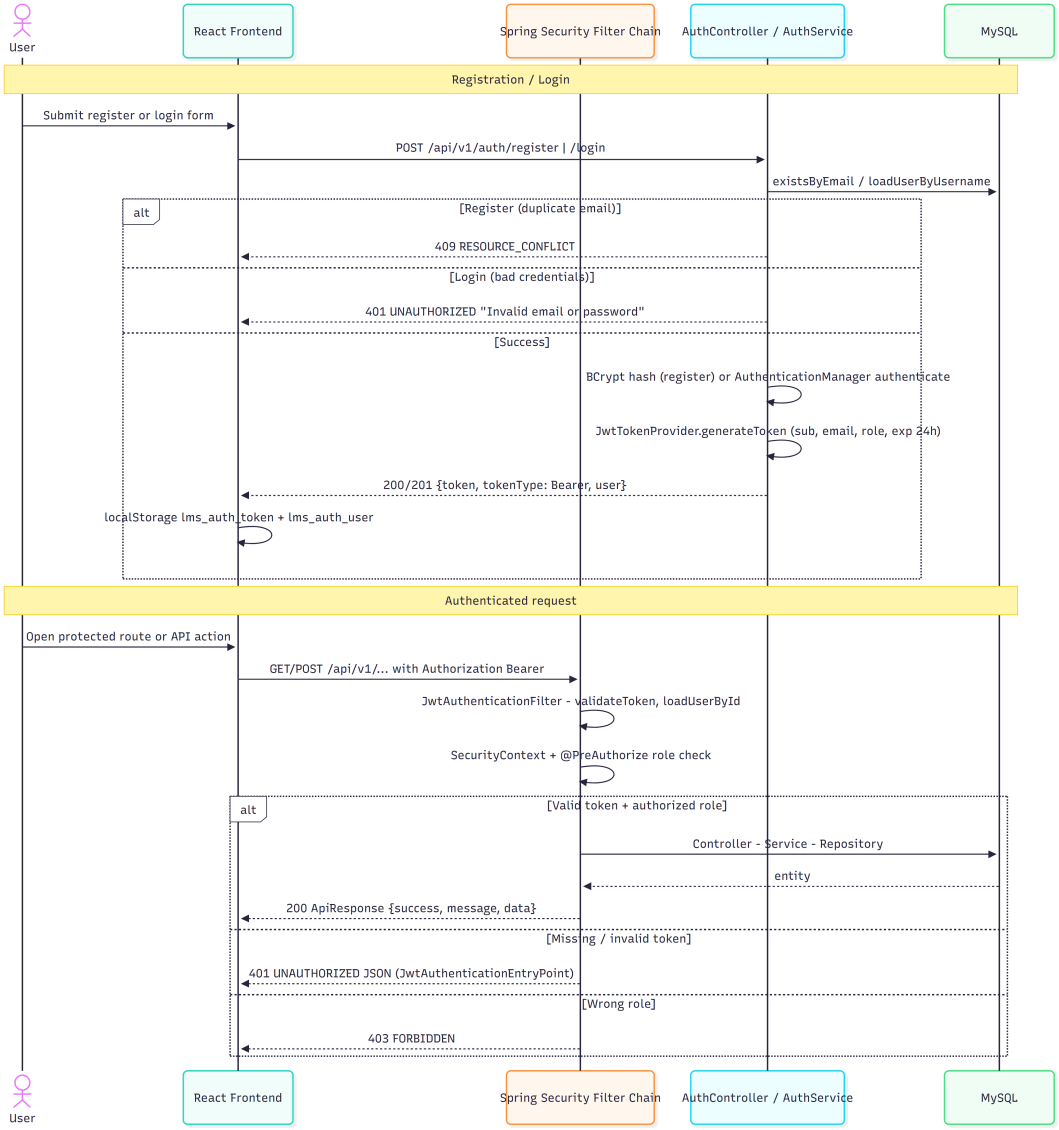


Figure 6: Authentication and authorisation control flow

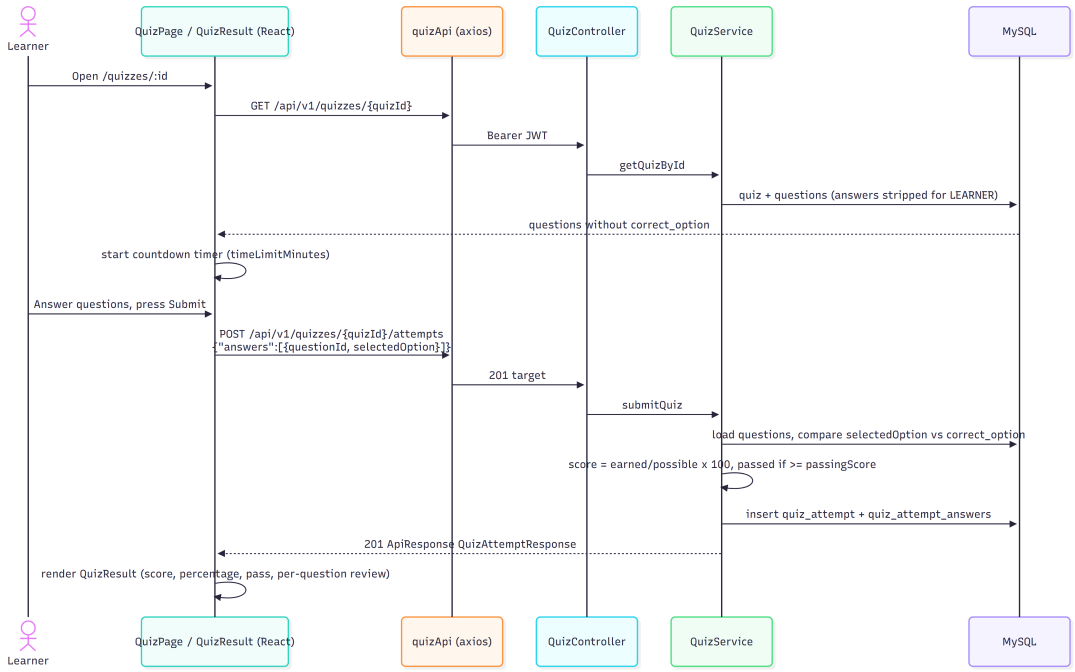


Figure 7: Quiz submission sequence – server-side authoritative scoring

11.4 API surface overview

Status: IMPLEMENTED

47 mapped endpoints across 11 controllers (detail in Section 16):

Table 38: API surface by domain

Domain	Endpoints	Base path	Public access?
Authentication	3	`/api/v1/auth`	Yes (register, login, logout)
Users	3	`/api/v1/users`	No (list is ADMIN-only)
Courses	5	`/api/v1/courses`	GET only
Categories	5	`/api/v1/categories`	GET only; writes ADMIN-only
Lessons	5	`/api/v1`	No
Enrollment	3	`/api/v1`	No
Progress	3	`/api/v1`	No
Quizzes & attempts	7	`/api/v1`	No
Discussions	4	`/api/v1`	No
Feedback	3	`/api/v1`	No
Certificates	2	`/api/v1`	No

11.5 Data contracts

Status: IMPLEMENTED

Listing 1: Standard success and error envelopes (ApiResponse.java, ErrorResponse.java)

```
{"success": true, "message": "Login successful",
```

```
 "data": {"token": "eyJhbGciOi...", "tokenType": "Bearer",
          "user": {"id": 1, "email": "learner@example.com", "role": "LEARNER"}}}

{"error": {"code": "VALIDATION_ERROR",
           "message": "Validation failed for one or more fields",
           "details": ["email: must be a well-formed email address"]}}
```

11.6 State management design

Status: IMPLEMENTED

- **Server state** lives in MySQL; the client refetches after mutations (no global client cache such as React Query).
- **Session state** is two `localStorage` keys (`lms_auth_token`, `lms_auth_user`) mirrored into React context on mount.
- **URL state** encodes classroom and workspace context (`?lesson=`, `?problem=`, `?redirect=`).
- **Component state** (loading/error/data) is local to pages; no Redux/Zustand store exists.

11.7 Error handling design

Status: IMPLEMENTED

1. Backend: controller/service throws typed exceptions → `GlobalExceptionHandler` maps to status + code + message (+ field details for validation).
2. Transport: Axios response interceptor normalises errors into `enhancedError` with `code`, `status`, `details`; network failures map to `NETWORK_ERROR`; timeouts use the 15 000 ms limit.
3. UI: `ErrorState` renders title/message/retry; per-action failures surface as antd `message.error` toasts.

Known gap: HTTP 401 responses do not clear the stored session or redirect to login (`frontend/src/api/client` – logged out users with stale tokens see repeated failures until they log out manually. **Status:** IMPLEMENTED (defect)

12 Low-Level Design

Low-level design documents the principal classes and components, their responsibilities, state, data access and dependencies. Symbols are cited exactly as they appear in source.

12.1 Backend – service layer detail

Status: IMPLEMENTED

Table 39: Service classes, responsibilities and key methods

Service	Responsibility	Key methods / rules
`AuthService`	Registration, login	`register` (409 on duplicate email, BCrypt, issues JWT, honours requested role); `login` (AuthenticationManager, 401 on bad credentials)
`UserService`	Profile read/update	`updateUser` (self or ADMIN; role change only by ADMIN); `getUserById` contains a no-op learner guard (defect)
`CourseService`	Catalogue CRUD	`getAllCourses` (Specification: search/category/status/instructor); `validateCourseOwnership` (admin or owning instructor); computed `totalLessons`, `totalEnrolled`
`CategoryService`	Category CRUD	Duplicate-name conflict check (case-insensitive)
`LessonService`	Lesson CRUD	Ownership via CourseService; default `orderIndex` = count+1; merges per-user completion flag
`EnrollmentService`	Enrol/drop/list	Duplicate `ENROLLED` → 409; re-enrol resets COMPLETED/DROPPED; `unenroll` sets DROPPED; self-or-ADMIN listing
`ProgressService`	Progress tracking	Find-or-create per user+lesson; `completed` and `percentage` kept consistent (100% ⇔ completed); auto-completes enrolment at 100%
`QuizService`	Quiz authoring & scoring	`includeAnswers` only for INSTRUCTOR/ADMIN; `submitQuiz` scores `earned/possible`, pass ≥ passing score; results scoped by role/ownership
`DiscussionService`	Threads & replies	Top-level listing (`parentIsNull`); delete restricted to author/ADMIN; no enrolment gate
`FeedbackService`	Ratings	Enrolment required; one row per user+course (409 otherwise); edit restricted to author/ADMIN
`CertificateService`	Credential issuance	Eligibility: enrolled + all lessons complete + every quiz passed; idempotent; sets enrolment COMPLETED; code `LMS-xxxxxxxx-NNNN`; grade hard-coded DISTINCTION

12.2 Backend – representative method detail

Status: IMPLEMENTED

Listing 2: Ownership check used by course/lesson/quiz mutations (CourseService.validateCourseOwnership)

```
private void validateCourseOwnership(Course course, UserPrincipal currentUser) {
    if (currentUser.getRole() == User.Role.ADMIN) return;
    boolean owner = currentUser.getRole() == User.Role.INSTRUCTOR
            && course.getInstructor().getId().equals(currentUser.getId());
    if (!owner) throw new ForbiddenException(
            "You do not have permission to modify this course");
}
```

Listing 3: Authoritative quiz scoring rule (QuizService.submitQuiz excerpt)

```
boolean isCorrect = question.getCorrectOption()
        .equalsIgnoreCase(submission.getSelectedOption().trim());
// earnedScore += points when correct; totalPossiblePoints over submitted answers
double scorePercentage = totalPossiblePoints > 0
        ? ((double) earnedScore / totalPossiblePoints) * 100.0 : 0.0;
boolean passed = scorePercentage >= quiz.getPassingScore(); // default 70
```

Design note: because `totalPossiblePoints` only accumulates points of questions present in the submission, unanswered questions do not reduce the percentage. **Status:** IMPLEMENTED (behaviour) – see Section 34.

12.3 Backend – persistence access

Status: IMPLEMENTED

- All 13 repositories extend `JpaRepository<Entity, Long>` with derived query methods (e.g. `findByEmail`, `existsByQuizIdAndUserIdAndPassedTrue`, `findByCourseIdAndParentIsNullOrderByCreat`
- No native queries, no `JdbcTemplate`, no specification API except `CourseRepository.findAll(Specificatio`
- Fetch strategy: `@ManyToOne` default EAGER for user/course links used in responses, LAZY for lesson/course back-references; collections (`Quiz.questions`, `QuizAttempt.answers`, `Discussion.replies`) are LAZY with cascade ALL + orphanRemoval.
- Timestamps set by `@PrePersist`/`@PreUpdate` callbacks inside entities.

12.4 Backend – data initialisation logic

Status: IMPLEMENTED

`backend/src/main/java/com/example/lms/config/DataInitializer.java` runs on every startup:

1. If `categoryRepository.count() == 0`: insert 4 categories (Web Development, Computer Science, Cloud & DevOps, Data Science & AI).
2. If `userRepository.count() == 0`: insert 3 users with BCrypt-encoded *hardcoded* passwords: `admin@example.com`/`admin123` (ADMIN), `instructor@example.com`/`password123` (INSTRUCTOR), `learner@example.com`/`password123` (LEARNER).

There is no profile or property guard; the seed runs on any empty database, including production. **Finding S-02, Section 23.**

12.5 Frontend – state and data access detail

Status: IMPLEMENTED

Table 40: Frontend state, storage and data access

Concern	Implementation	State/keys
Session	`frontend/src/context/AuthContext.jsx`	`lms_auth_token`, `lms_auth_user`; `isLoading` restore gate
Derived auth	`role`, `isAuthenticated`, `isLearner/isInstructor/isAdmin`	Computed on each context value build
Route guards	`frontend/src/routes/ProtectedRoute.jsx`	Loading → spinner; unauth → `/login?redirect=`; wrong role → 403 card
HTTP	`frontend/src/api/client.js`	base URL `VITE_API_BASE_URL` or `http://localhost:8080`; 15 s timeout; Bearer injection; envelope unwrapping
Page data	Local `useState` + `useEffect` fetches	loading/error/data triples per page
URL query	`useSearchParams`	`lesson`, `problem`, `redirect`

12.6 Frontend – key component behaviours

Status: IMPLEMENTED

Table 41: Representative component logic

Component	Behaviour
`frontend/src/components/LessonPlayer.jsx`	Detects `.pdf` / YouTube/Vimeo / `.mp4` → embed; other URL → external-resource button; none → text lesson); `recordLessonProgress` posts `{completed: true, completionPercentage: 100, watchTimeSeconds}`
`frontend/src/components/QuizPage.jsx`	One question at a time with palette; countdown from `timeLimitMinutes`; confirmation modal on partial submit; delegates to `QuizResult`
`frontend/src/components/practice/CodeEditor.jsx`	Builds practice URL (`compilerEdit` + `listenToEvents`); injects starter code via `postMessage` on load, after 400 ms, and on button click
`frontend/src/components/EnrollmentButton.jsx`	Unauth → login redirect; instructor → blocked; enrolled → Continue Learning (+ optional Drop); else enrol via API
`frontend/src/components/CertificateCard.jsx`	Renders learner name/course/grade/date; `window.print()` for PDF saving
`frontend/src/components/InstructorDashboard.jsx`	Client-side tabs for Lessons/Assessments; modals for course/lesson/quiz authoring; submissions modal with pass/fail badges

12.7 Dependencies (internal)

Status: IMPLEMENTED

Listing 4: Internal dependency direction (no cycles beyond declared parent-child)

```
Controller --> Service --> Repository --> Entity
Service   --> other Service (CourseService <-- Lesson/Quiz/Certificate/Enrollment)
Service   --> Security (UserPrincipal for identity/role)
Component --> api module --> client (axios) --> backend
Page      --> components --> api modules
Context   --> authApi ; Pages --> useAuth()
```

13 Frontend Architecture

13.1 Application entry and providers

Status: IMPLEMENTED

Listing 5: Bootstrap chain (frontend/src/main.jsx, frontend/src/App.jsx)

```
main.jsx
  -> <StrictMode> -> <AuthProvider> -> <BrowserRouter> -> <ConfigProvider antdTheme>
       -> <AppShell> (Navbar + Sidebar/Drawer + main + Footer)
            -> <AppRoutes />
```

The antd theme in `frontend/src/App.jsx` injects the design tokens (black primary, 4px radius, 40px control height, custom component overrides) described in Section 22.

13.2 Routing

Status: IMPLEMENTED

`frontend/src/routes/AppRoutes.jsx` declares 17 routes:

Table 42: Frontend route table

Path	Component	Guard
`/`	HomePage	public
`/login`	LoginPage	public (`?redirect=` honoured)
`/register`	RegisterPage	public
`/courses`	CourseCatalogPage	public
`/courses/:id`	CourseDetailPage (CourseDetails)	public
`/practice`	PracticePage	public
`/courses/:courseId/practice`	CoursePracticePage	public
`/courses/:courseId/learn`	LearningPage	authenticated
`/quizzes/:id`	QuizTakingPage (QuizPage)	authenticated
`/learner`, `/learner/certificates`	LearnerDashboard	LEARNER, ADMIN
`/instructor`, `/instructor/courses/new`	InstructorDashboard	INSTRUCTOR, ADMIN
`/admin`, `/admin/users`, `/admin/categories`	AdminDashboard	ADMIN
`/profile`	ProfilePage	authenticated
`*`	NotFoundPage	–

13.3 Guard semantics

Status: IMPLEMENTED (with known gaps)

`frontend/src/routes/ProtectedRoute.jsx`:

- `isLoading` → full-page `LoadingState` ("Authenticating…").
- Not authenticated → redirect to `/login?redirect=<path>` (the original query string is dropped – defect KI-10).
- Authenticated but role not in `allowedRoles` → inline 403 card ("Access Restricted") with a full-page-home button.
- Guards are usability controls only; the backend enforces authorisation (`CONTRACT.md` section 13).

13.4 Navigation structure

Status: IMPLEMENTED

- **Navbar** (sticky): brand images, "Create Course" link (INSTRUCTOR only), auth-aware dropdown (Dashboard, Profile, Log Out) with avatar and role badge.
- **Sidebar** (desktop, authenticated): role-specific `NavLink` sets – ADMIN: `/admin`, `/courses`, `/admin/users`, `/admin/categories`, `/profile`; INSTRUCTOR: `/instructor`, `/courses`, `/instructor/courses/new`, `/profile`; LEARNER: `/learner`, `/courses`, `/learner/certificates`, `/profile`. Mobile uses an antd Drawer (width 320).
- **Footer**: five columns (brand, Learn, Profile, Account, Practice). Three links point to routes that do not exist (`/learner/enrollments`, `/learner/progress`, `/settings`) – defect KI-01.

13.5 API layer

Status: IMPLEMENTED

Table 43: Frontend API modules and the endpoints they call

Module	Endpoints invoked
authApi	POST /api/v1/auth/register, POST /api/v1/auth/login, POST /api/v1/auth/logout
userApi	GET /api/v1/users/{id}, PUT /api/v1/users/{id}, GET /api/v1/users
categoryApi	GET/POST/PUT/DELETE /api/v1/categories[...], GET /api/v1/categories/{id}
courseApi	GET /api/v1/courses?search&categoryId&status&instructorId, GET/PUT/DELETE /api/v1/courses/{id}, POST /api/v1/courses
enrollmentApi	POST /api/v1/courses/{id}/enroll, DELETE .../enroll?userId=, GET /api/v1/users/{id}/enrollments
lessonApi	GET/POST /api/v1/courses/{id}/lessons, GET/PUT/DELETE /api/v1/lessons/{id}
progressApi	GET /api/v1/courses/{id}/progress, POST/PATCH /api/v1/lessons/{id}/progress
quizApi	GET /api/v1/courses/{id}/quizzes, GET/PUT/DELETE /api/v1/quizzes/{id}, POST .../attempts, GET .../results, POST /api/v1/courses/{id}/quizzes
discussionApi	GET/POST /api/v1/courses/{id}/discussions, POST /api/v1/discussions/{id}/replies, DELETE /api/v1/discussions/{id}
feedbackApi	GET/POST /api/v1/courses/{id}/feedback, PUT /api/v1/feedback/{id}
certificateApi	GET/POST /api/v1/courses/{id}/certificate

Every path above was cross-checked against the Spring controllers – there are no phantom frontend calls. The shared `frontend/src/api/client.js` attaches the Bearer token, unwraps envelopes, and normalises errors (Section 11).

13.6 Component inventory

Status: IMPLEMENTED

Table 44: Component inventory (selected, with purpose)

Component	Purpose
AppShell / Navbar / Sidebar / Footer	Application chrome and role-aware navigation
LearnerDashboard	KPI tiles, active/completed tabs, certificates list
InstructorDashboard	Course table, content manager tabs (Lessons, Assessments), authoring modals, submissions modal
AdminDashboard	KPI tiles, user directory, category CRUD, course moderation
CourseCard / CourseList / CourseDetails	Catalogue presentation and detail tabs (Syllabus, Assessments, Practice, Discussions, Reviews)
EnrollmentButton / ProgressBar	Enrolment interaction and progress display
LessonPlayer / LearningPage	Classroom: media player, curriculum sidebar, certificate modal
QuizPage / QuizResult / QuizCard	Timed assessment flow and review
DiscussionList / DiscussionItem	Threads with recursive replies (UI depth ≤ 2)
FeedbackForm	Ratings, average, star distribution, reviews
CertificateCard	Printable credential
practice/*	Problem list, workspace, statement panel, OneCompiler editor, course practice tab
common/*	LoadingState, ErrorState, EmptyState (tested state primitives)
ui/*	Card, GradientRibbon, SectionHeader, Badge, StatTile, WordmarkBanner (design-system kit)

13.7 Practice module structure

Status: IMPLEMENTED

`frontend/src/components/practice/` implements a self-contained feature: data access strictly through `fetchPracticeProblems` in `frontend/src/data/practiceData.js` (a local dataset of 7 problems across Python, JavaScript and Java, three of them bound to seeded course ids), rendering problem statements and a OneCompiler iframe. The file header documents the intended swap boundary: replace the local promise with `apiClient.get(...)` when a backend practice API exists. **Status:** IMPLEMENTED (client) / NOT IMPLEMENTED (server).

13.8 Build pipeline

Status: IMPLEMENTED

Listing 6: Verified frontend commands (frontend/package.json)

```
npm install
npm run dev        # vite, port 3000, /api proxy to :8080
```

```
npm run build      # vite build -> dist/
npm run preview    # vite preview
npm test           # vitest run (39 tests)
npm run test:watch # vitest watch
```

There is no lint or format script; no ESLint/Prettier configuration exists in the repository. **Status:** NOT IMPLEMENTED (linting)

14 Backend Architecture

14.1 Layered structure

Status: IMPLEMENTED

The backend follows the canonical layering mandated by `CONTRACT.md` section 7:

`Controller` → `Service` → `Repository` → `Database`

Controllers never touch repositories directly (verified: all persistence calls go through service classes); DTOs isolate the wire format from JPA entities; security components sit *beside* the layers in the servlet filter chain.

14.2 Component inventory

Table 45: Backend component counts

Component type	Count	Location
REST controllers	11	`backend/src/main/java/com/example/lms/controller`
Service classes	11	`backend/src/main/java/com/example/lms/service/`
JPA repositories	13	`backend/src/main/java/com/example/lms/repository`
Entities	13	`backend/src/main/java/com/example/lms/entity/`
Request DTOs	15	`backend/src/main/java/com/example/lms/dto/reques`
Response DTOs	15	`backend/src/main/java/com/example/lms/dto/respon`
Security components	5	`backend/src/main/java/com/example/lms/security/`
Exception types	7 (1 advice + 6)	`backend/src/main/java/com/example/lms/exception/`
Configuration classes	3	`backend/src/main/java/com/example/lms/config/`

14.3 Request lifecycle

Status: IMPLEMENTED

1. **Container** receives HTTP request on port `PORT` (8080 default).
2. **JwtAuthenticationFilter** (`OncePerRequestFilter`) extracts `Authorization: Bearer` token, validates signature/expiry, loads the user by id, and populates `SecurityContextHolder`. Failures are logged and the request continues unauthenticated.
3. **Spring Security authorisation** applies URL matchers (`permitAll` / `hasRole`) or throws `AccessDeniedException`.
4. **DispatcherServlet** routes to a controller method; `@Valid` triggers Bean Validation.
5. **Method security** evaluates `@PreAuthorize` expressions (`hasRole('ADMIN')`, `hasAnyRole('INSTRUCTOR','A`

6. **Service** executes business rules (ownership checks, eligibility, scoring) inside a JPA transaction.
7. **Repository** persists/queries; entities map to the 13 tables.
8. **Response** DTOs are serialised inside the `ApiResponse` envelope; exceptions are translated by `GlobalExceptionHandler`.

14.4 Security components in detail

Table 46: Security component responsibilities

Class	Behaviour (verified)
`security/JwtTokenProvider.java`	Builds HMAC key from Base64-decoded `app.jwt.secret`; issues claims `sub` (user id), `email`, `role`, `iat`, `exp`; `validateToken` catches `JwtException` and returns false
`security/JwtAuthenticationFilter.java`	Reads `Bearer` token (prefix `"Bearer "`), validates, loads `UserPrincipal` by id, sets authentication; never blocks the chain on failure
`security/CustomUserDetailsService.java`	Loads user by email or id; wraps as `UserPrincipal`
`security/UserPrincipal.java`	Exposes single authority `ROLE_LEARNER|INSTRUCTOR|ADMIN`; `getUsername()` returns email
`security/JwtAuthenticationEntryPoint.java`	Returns 401 JSON `{error: {code: "UNAUTHORIZED", ...}}` for unauthenticated access

14.5 DTO and mapping strategy

Status: IMPLEMENTED

- Request DTOs carry validation annotations (`@NotBlank`, `@Email`, `@Size`, `@Min`/`@Max`); example: `dto/request/RegisterRequest.java` (name 2–100, password 6–100).
- Response DTOs are built by explicit static factories (e.g. `UserResponse.fromEntity`) – no MapStruct/modelMapper dependency.
- Entities are never serialised directly; the envelope adds `success`/`message`/`data`.
- Sensitive fields are excluded by construction (`password` never appears in `UserResponse`); question DTOs strip `correctOption` for learners via the `includeAnswers` flag.

14.6 Exception taxonomy

Status: IMPLEMENTED

Table 47: Exception types and HTTP mapping (from GlobalExceptionHandler)

Exception	HTTP	Error code
`ResourceNotFoundException`	404	`RESOURCE_NOT_FOUND`
`ResourceConflictException`	409	`RESOURCE_CONFLICT`
`ForbiddenException`	403	`FORBIDDEN`
`UnauthorizedException`	401	`UNAUTHORIZED`
`BadRequestException`	400	`VALIDATION_ERROR` (or custom)
`MethodArgumentNotValidException`	400	`VALIDATION_ERROR` + per-field `details`
`AccessDeniedException` (Spring)	403	`FORBIDDEN`
`AuthenticationException`	401	`UNAUTHORIZED`
Catch-all `Exception`	500	`INTERNAL_SERVER_ERROR` (generic message, logged)

14.7 Configuration and bootstrap

Status: IMPLEMENTED

- `backend/src/main/resources/application.yml` – port, datasource (`DB_URL` with local default), `ddl-auto: update`, JWT secret/expiry with committed defaults, springdoc paths, log levels.
- `backend/src/main/resources/application-test.yml` – H2 in-memory (`MODE=MySQL`), `create-drop`, 1-hour JWT, quiet logging.
- `config/DataInitializer.java` – `CommandLineRunner` that seeds 4 categories when the category table is empty and 3 demo users (with BCrypt-encoded, *hardcoded* passwords) when the user table is empty. No profile guard applies. **See Section 23 for the associated findings.**
- `config/OpenApiConfig.java` – OpenAPI 3 document *"Learning Management System (LMS) API"* v1.0.0, MIT licence string, bearer security scheme, contact block for the backend workstream.

14.8 Build and packaging

Status: IMPLEMENTED

Listing 7: Verified backend build and run commands (backend/pom.xml, README.md)

```
cd backend
./mvnw spring-boot:run      # development
./mvnw test                 # run backend tests
./mvnw clean package        # build lms-backend-1.0.0.jar
```

The Docker image runs the same artifact (`java -jar app.jar`) under `eclipse-temurin:21-jre-alpine`. Tests skip during container build (`-DskipTests`).

15 Database Design

15.1 Technology and conventions

Status: IMPLEMENTED

Table 48: Database technology and design conventions

Aspect	Fact (evidence)
Engine	MySQL 8, InnoDB, `utf8mb4_unicode_ci` (`database/schema.sql`)
Test engine	H2 in-memory, `MODE=MySQL`, `ddl-auto: create-drop` (`backend/src/main/resources/application-test.yml`)
Schema authority	`database/schema.sql` (hand-authored) + Hibernate `ddl-auto: update` at startup
Identifier strategy	`BIGINT AUTO_INCREMENT` primary keys (no UUIDs)
Timestamps	`created_at`/`updated_at` defaults, `ON UPDATE CURRENT_TIMESTAMP`
Soft delete	Enrolments use status `DROPPED`; courses/users are hard-deleted
Naming	snake_case tables/columns; JPA maps via naming strategy
Table count	13 tables

15.2 Entity-relationship diagram

Status: IMPLEMENTED

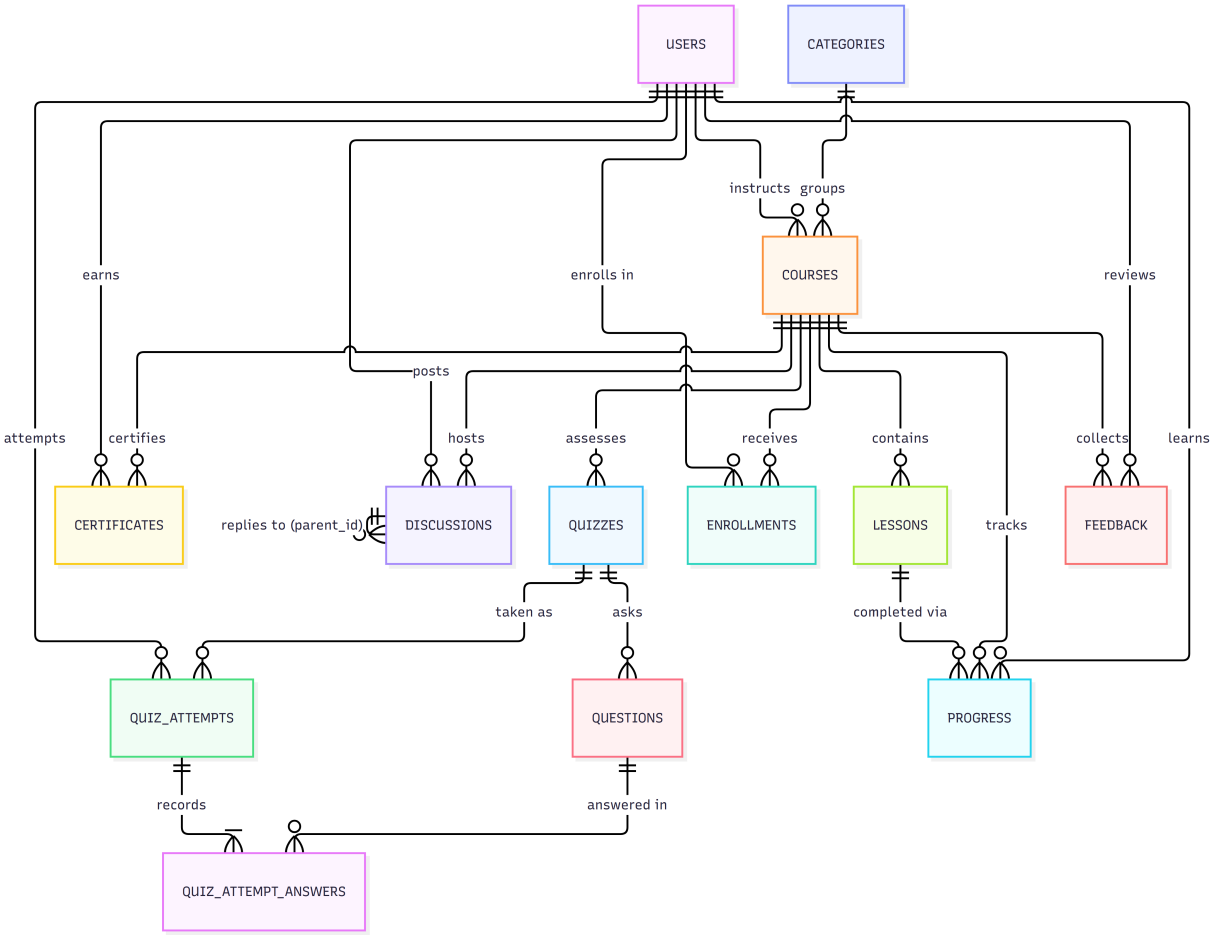


Figure 8: Entity-relationship diagram of the 13 LMS tables

15.3 Tables

Status: IMPLEMENTED

15.3.1 users

Listing 8: users table (database/schema.sql)

```
id BIGINT PK AUTO_INCREMENT
name VARCHAR(100) NOT NULL
email VARCHAR(150) NOT NULL UNIQUE          -- idx_users_email
password VARCHAR(255) NOT NULL              -- BCrypt hash
role VARCHAR(30) NOT NULL DEFAULT 'LEARNER' -- idx_users_role
bio TEXT NULL
avatar_url VARCHAR(512) NULL
created_at, updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
```

15.3.2 categories

```
id BIGINT PK
name VARCHAR(100) NOT NULL UNIQUE
description TEXT NULL
created_at, updated_at TIMESTAMP ...
```

15.3.3 courses

```
id BIGINT PK
title VARCHAR(255) NOT NULL
description TEXT NULL
category_id BIGINT NULL  -> categories(id) ON DELETE SET NULL   -- fk_courses_category
instructor_id BIGINT NOT NULL -> users(id) ON DELETE CASCADE   -- fk_courses_instructor
status VARCHAR(30) NOT NULL DEFAULT 'DRAFT'                    -- idx_courses_status
thumbnail_url VARCHAR(512) NULL
-- indexes: idx_courses_instructor, idx_courses_category
```

15.3.4 lessons

```
id BIGINT PK
course_id BIGINT NOT NULL -> courses(id) ON DELETE CASCADE      -- fk_lessons_course
title VARCHAR(255) NOT NULL
description TEXT NULL
content_url VARCHAR(512) NULL        -- video / external resource
content_text LONGTEXT NULL           -- written lesson body
order_index INT NOT NULL DEFAULT 1   -- idx_lessons_order (course_id, order_index)
duration_minutes INT NOT NULL DEFAULT 0
created_at, updated_at TIMESTAMP ...
```

15.3.5 enrollments

```
id BIGINT PK
user_id BIGINT NOT NULL -> users(id) ON DELETE CASCADE
course_id BIGINT NOT NULL -> courses(id) ON DELETE CASCADE
status VARCHAR(30) NOT NULL DEFAULT 'ENROLLED'  -- ENROLLED|COMPLETED|DROPPED
enrolled_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
completed_at TIMESTAMP NULL
UNIQUE KEY uk_user_course_enrollment (user_id, course_id)
-- indexes: idx_enrollments_user, idx_enrollments_course, idx_enrollments_status
```

15.3.6 progress

```
id BIGINT PK
user_id BIGINT NOT NULL -> users(id) ON DELETE CASCADE
course_id BIGINT NOT NULL -> courses(id) ON DELETE CASCADE
lesson_id BIGINT NOT NULL -> lessons(id) ON DELETE CASCADE
completed BOOLEAN NOT NULL DEFAULT FALSE
watch_time_seconds INT NOT NULL DEFAULT 0
completion_percentage DOUBLE NOT NULL DEFAULT 0.0
updated_at TIMESTAMP ...
UNIQUE KEY uk_user_lesson_progress (user_id, lesson_id)
INDEX idx_progress_user_course (user_id, course_id)
```

15.3.7 quizzes

```
id BIGINT PK
course_id BIGINT NOT NULL -> courses(id) ON DELETE CASCADE
title VARCHAR(255) NOT NULL
description TEXT NULL
passing_score INT NOT NULL DEFAULT 70
time_limit_minutes INT NOT NULL DEFAULT 30
created_at, updated_at TIMESTAMP ...
INDEX idx_quizzes_course (course_id)
```

15.3.8 questions

```
id BIGINT PK
quiz_id BIGINT NOT NULL -> quizzes(id) ON DELETE CASCADE
question_text TEXT NOT NULL
option_a..option_d VARCHAR(500) NOT NULL      -- four fixed options
correct_option VARCHAR(10) NOT NULL           -- 'A'..'D'
points INT NOT NULL DEFAULT 10
explanation TEXT NULL
created_at, updated_at TIMESTAMP ...
INDEX idx_questions_quiz (quiz_id)
```

15.3.9 quiz_attempts

and

15.3.10 quiz_attempt_answers

```
quiz_attempts:
  id BIGINT PK
  quiz_id BIGINT NOT NULL -> quizzes(id) ON DELETE CASCADE
  user_id BIGINT NOT NULL -> users(id) ON DELETE CASCADE
  score INT DEFAULT 0, total_points INT DEFAULT 0, passed BOOLEAN DEFAULT FALSE
  started_at TIMESTAMP NOT NULL, submitted_at TIMESTAMP NULL
  indexes: idx_quiz_attempts_user, idx_quiz_attempts_quiz

quiz_attempt_answers:
  id BIGINT PK
  attempt_id BIGINT NOT NULL -> quiz_attempts(id) ON DELETE CASCADE
  question_id BIGINT NOT NULL -> questions(id) ON DELETE CASCADE
  selected_option VARCHAR(10) NOT NULL
  is_correct BOOLEAN DEFAULT FALSE, points_awarded INT DEFAULT 0
  index: idx_attempt_answers_attempt
```

15.3.11 discussions

```
id BIGINT PK
course_id BIGINT NOT NULL -> courses(id) ON DELETE CASCADE
user_id BIGINT NOT NULL -> users(id) ON DELETE CASCADE
parent_id BIGINT NULL -> discussions(id) ON DELETE CASCADE   -- self reference (threads
    /replies)
title VARCHAR(255) NULL       -- NULL for replies
content TEXT NOT NULL
created_at, updated_at TIMESTAMP ...
indexes: idx_discussions_course, idx_discussions_parent
```

15.3.12 feedback

```
id BIGINT PK
course_id BIGINT NOT NULL -> courses(id) ON DELETE CASCADE
user_id BIGINT NOT NULL -> users(id) ON DELETE CASCADE
rating INT NOT NULL CHECK (rating >= 1 AND rating <= 5)
comment TEXT NULL
created_at, updated_at TIMESTAMP ...
UNIQUE KEY uk_user_course_feedback (user_id, course_id)
INDEX idx_feedback_course (course_id)
```

15.3.13 certificates

```
id BIGINT PK
```

```
certificate_code VARCHAR(64) NOT NULL UNIQUE    -- idx_certificates_code
user_id BIGINT NOT NULL -> users(id) ON DELETE CASCADE
course_id BIGINT NOT NULL -> courses(id) ON DELETE CASCADE
issue_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
grade VARCHAR(50) DEFAULT 'PASS'                -- runtime value is 'DISTINCTION'
certificate_url VARCHAR(512) NULL               -- API path to the certificate
UNIQUE KEY uk_user_course_certificate (user_id, course_id)
```

15.4 Relationship summary

Status: IMPLEMENTED

Table 49: Foreign-key relationships and cardinalities

Child	Parent	Cardinality	On delete
courses.category_id	categories	optional many–one	SET NULL
courses.instructor_id	users	many–one (required)	CASCADE
lessons.course_id	courses	many–one	CASCADE
enrollments.user_id / course_id	users / courses	many–many (associative)	CASCADE
progress.user_id / course_id / lesson_id	users / courses / lessons	many–one per dimension	CASCADE
quizzes.course_id	courses	many–one	CASCADE
questions.quiz_id	quizzes	many–one	CASCADE
quiz_attempts.quiz_id / user_id	quizzes / users	many–one	CASCADE
quiz_attempt_answers.attempt_id / question_id	quiz_attempts / questions	many–one	CASCADE
discussions.parent_id	discussions	self (thread/reply)	CASCADE
feedback.course_id / user_id	courses / users	many–one	CASCADE
certificates.user_id / course_id	users / courses	many–one	CASCADE

15.5 Indexes and constraints

Status: IMPLEMENTED

- **20 secondary indexes** declared in `database/schema.sql` (email, role, instructor, category, status, course lookups, user/course pairs, attempt/answer lookups, discussion parent, certificate code).
- **4 unique business keys:** `uk_user_course_enrollment`, `uk_user_lesson_progress`, `uk_user_course_fee` `uk_user_course_certificate` – these enforce "one enrolment / one progress row / one review / one certificate" at the database level.
- **Check constraint:** `feedback.rating` between 1 and 5 (schema-level; additionally enforced

by `@Min(1)`/`@Max(5)` on the DTO).
- **Referential deletion:** child rows cascade with their parents; removing a category only detaches courses.

15.6 Views, stored procedures, functions, triggers

Status: NOT APPLICABLE (verified)

`database/schema.sql` contains no `VIEW`, `PROCEDURE`, `FUNCTION` or `TRIGGER` statements; all derived values (progress percentages, counts, scores) are computed in the application layer.

15.7 Row-level security and database access control

Status: NOT IMPLEMENTED / NOT APPLICABLE

MySQL has no RLS feature; the repository configures no dedicated database roles or grants (the app connects with credentials from the datasource configuration, defaulting to `root`). All row-level enforcement happens in Java services. **Status:** RECOMMENDED a least-privilege application DB user for production.

15.8 Migrations and schema evolution

Status: PARTIAL
- No Flyway or Liquibase dependency exists in `backend/pom.xml`; there are no versioned migration files.
- Schema is created by `database/schema.sql` (manual import, documented in `README.md`) and then kept in sync by Hibernate `ddl-auto: update`, which can add tables/columns but never alters or drops them.
- `database/seed.sql` and `database/dummy.sql` provide `INSERT IGNORE` sample data (demo users with BCrypt hash for `password123`, 2–3 sample courses, lessons, quizzes, questions).

Risk: schema drift between hand-written SQL and entity definitions is possible and untested in CI (no pipeline). See Section 35.

15.9 Entity/schema parity notes

Status: IMPLEMENTED (observations)
- All 13 `@Entity` classes map one-to-one to the 13 tables (verified: `@Table` names match).
- The runtime `grade` value written by `CertificateService` is `DISTINCTION` while the column default is `PASS` – cosmetic inconsistency documented in Section 34.
- The `progress` table has no `created_at` (only `updated_at`) – matches the entity.

16 API Documentation

16.1 Base conventions

Status: IMPLEMENTED

All REST endpoints are versioned under `/api/v1`. JSON request/response bodies use DTOs in `backend/src/main/java/com/example/lms/dto/`. Success envelopes use `backend/src/main/java/com/exampl` errors use `backend/src/main/java/com/example/lms/dto/response/ErrorResponse.java`. Interactive docs are served by springdoc-openapi 2.6.0 (`backend/src/main/java/com/example/lms/config/OpenA` OpenAPI JSON at `/v3/api-docs` and Swagger UI at `/swagger-ui.html` or `/swagger-ui/index.html`.

16.2 Endpoint inventory (47 mapped endpoints, 11 controllers)

Status: IMPLEMENTED

Table 50: Controller to endpoint-group mapping (verified in controller sources)

Controller	Endpoints
controller/AuthController.java	POST /api/v1/auth/register, POST /api/v1/auth/login, POST /api/v1/auth/logout
controller/UserController.java	GET /api/v1/users, GET /api/v1/users/\{id\}, PUT /api/v1/users/\{id\}
controller/CourseController.java	GET /api/v1/courses, GET /api/v1/courses/\{id\}, POST /api/v1/courses, PUT /api/v1/courses/\{id\}, DELETE /api/v1/courses/\{id\}
controller/EnrollmentController.java	POST /api/v1/courses/\{courseId\}/enroll, DELETE /api/v1/courses/\{courseId\}/enroll, GET /api/v1/users/\{userId\}/enrollments
controller/LessonController.java	GET /api/v1/courses/\{courseId\}/lessons, GET /api/v1/lessons/\{lessonId\}, POST /api/v1/courses/\{courseId\}/lessons, PUT /api/v1/lessons/\{lessonId\}, DELETE /api/v1/lessons/\{lessonId\}
controller/ProgressController.java	GET /api/v1/courses/\{courseId\}/progress, POST /api/v1/lessons/\{lessonId\}/progress, PATCH /api/v1/lessons/\{lessonId\}/progress
controller/QuizController.java	Quiz CRUD plus POST /api/v1/quizzes/\{quizId\}/attempts, GET /api/v1/quizzes/\{quizId\}/results
controller/DiscussionController.java	GET/POST /api/v1/courses/\{courseId\}/discussions, replies, delete
controller/FeedbackController.java	GET/POST /api/v1/courses/\{courseId\}/feedback, PUT /api/v1/feedback/\{feedbackId\}
controller/CertificateController.java	GET/POST /api/v1/courses/\{courseId\}/certificate
controller/CategoryController.java	Category CRUD under /api/v1/categories

16.3 Error model

Status: IMPLEMENTED

Canonical codes: `VALIDATION_ERROR` (400), `UNAUTHORIZED` (401), `FORBIDDEN` (403), `RESOURCE_NOT_FOUND` (404), `RESOURCE_CONFLICT` (409, e.g. duplicate enrollment), `INVALID_OPERATION` (422), `INTERNAL_SERVER_ERROR` (500). Stack traces and credentials are never returned. The frontend normalises these in `frontend/src/api/client.js`.

17 Authentication

17.1 Mechanism

Status: IMPLEMENTED

Stateless JWT authentication. Registration and login are handled by `backend/src/main/java/com/example/` and `backend/src/main/java/com/example/lms/service/AuthService.java`. Passwords are hashed with BCrypt; on success the backend issues a signed JWT (JJWT 0.12.6, secret `JWT_SECRET`, expiry `JWT_EXPIRATION=86400000` per `.env.example`). The frontend stores the token in `localStorage` (`frontend/src/context/AuthContext.jsx`) and attaches `Authorization: Bearer <token>` via an Axios interceptor (`frontend/src/api/client.js`).

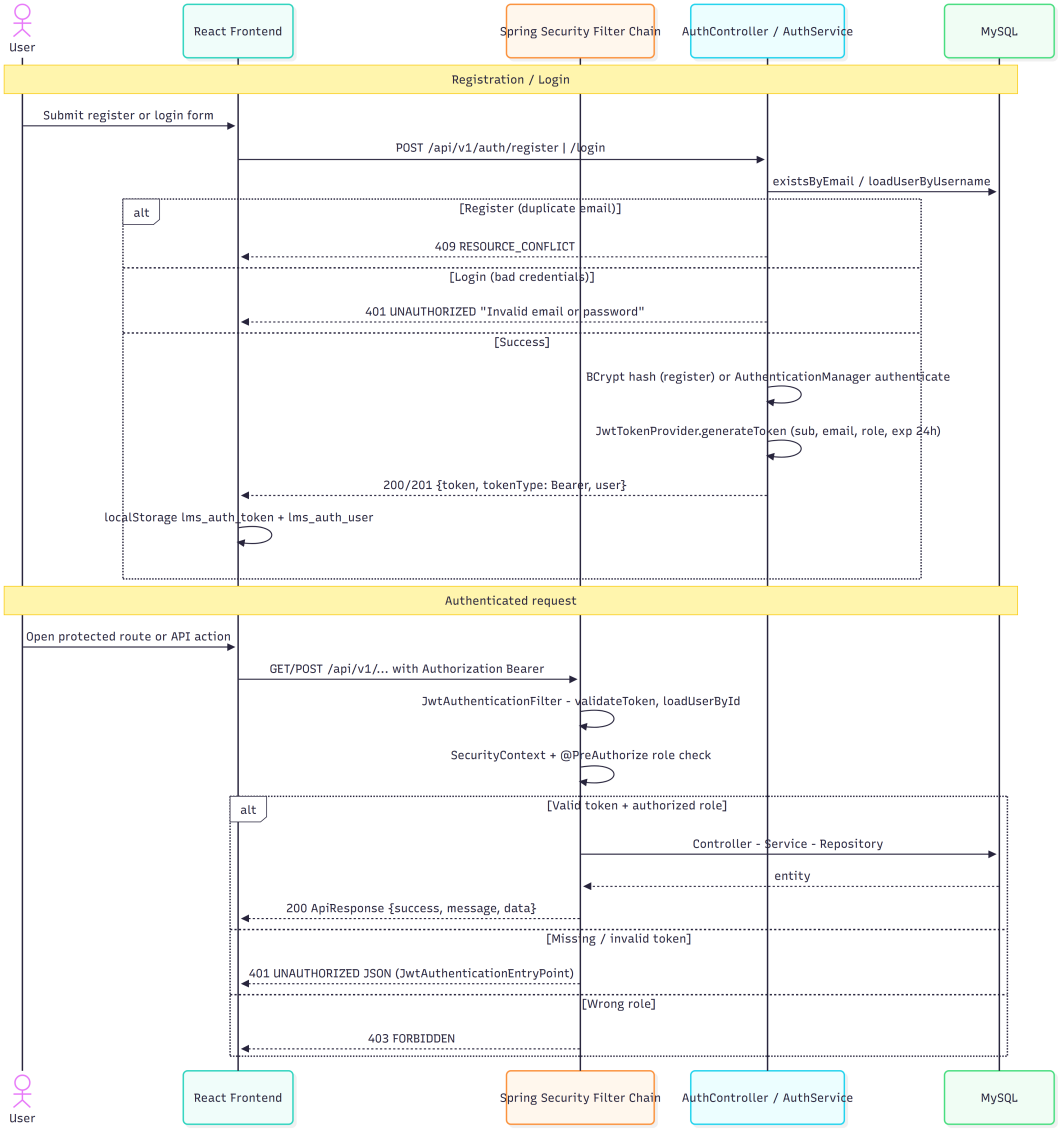


Figure 9: Authentication flow – register, login, JWT issuance, per-request validation

17.2 Validation chain

Status: IMPLEMENTED

Every request passes through the Spring Security filter chain (`backend/src/main/java/com/example/lms/co`

plus `backend/src/main/java/com/example/lms/security/`): token signature and expiry are validated, the user principal is loaded, and method-level rules are applied. Invalid or expired tokens yield 401. There is no server-side session store and no token revocation list – logout is client-side token removal plus a `POST /api/v1/auth/logout` acknowledgement.

17.3 Limitations

Status: IMPLEMENTED (gaps recorded)

No password recovery, no email verification, no refresh-token rotation, and no lockout after repeated failures. Seed users created by `backend/src/main/java/com/example/lms/config/DataInitializer.j` use hardcoded passwords (finding S-02, Section 23).

18 Authorization

18.1 Model

Status: IMPLEMENTED

Role-based access control with three roles stored in `users.role` (`database/schema.sql`, default `LEARNER`): `LEARNER`, `INSTRUCTOR`, `ADMIN`. Enforcement is server-side via Spring Security method annotations; the frontend mirrors the same rules for UX only through `frontend/src/routes/ProtectedR` and `frontend/src/routes/AppRoutes.jsx` (route guards are not a security boundary – CONTRACT.md section 13).

18.2 Authorization matrix

Status: IMPLEMENTED

Table 51: Authorization matrix (backend-enforced; frontend mirrors for UX)

Feature	Learner	Instructor	Admin
Browse / view courses	Yes	Yes	Yes
Enroll in courses	Yes	–	Yes
Access lessons / track own progress	Yes	Yes (own)	Yes
Take quiz / view own results	Yes	–	Yes
Create / edit / delete own course	–	Yes	Yes
Manage questions and quizzes	–	Yes	Yes
View learner performance	–	Yes (own courses)	Yes
Participate in discussions	Yes	Yes	Yes
Submit feedback	Yes	–	–
Manage users / instructors / all courses	–	–	Yes
Access admin dashboard	–	–	Yes

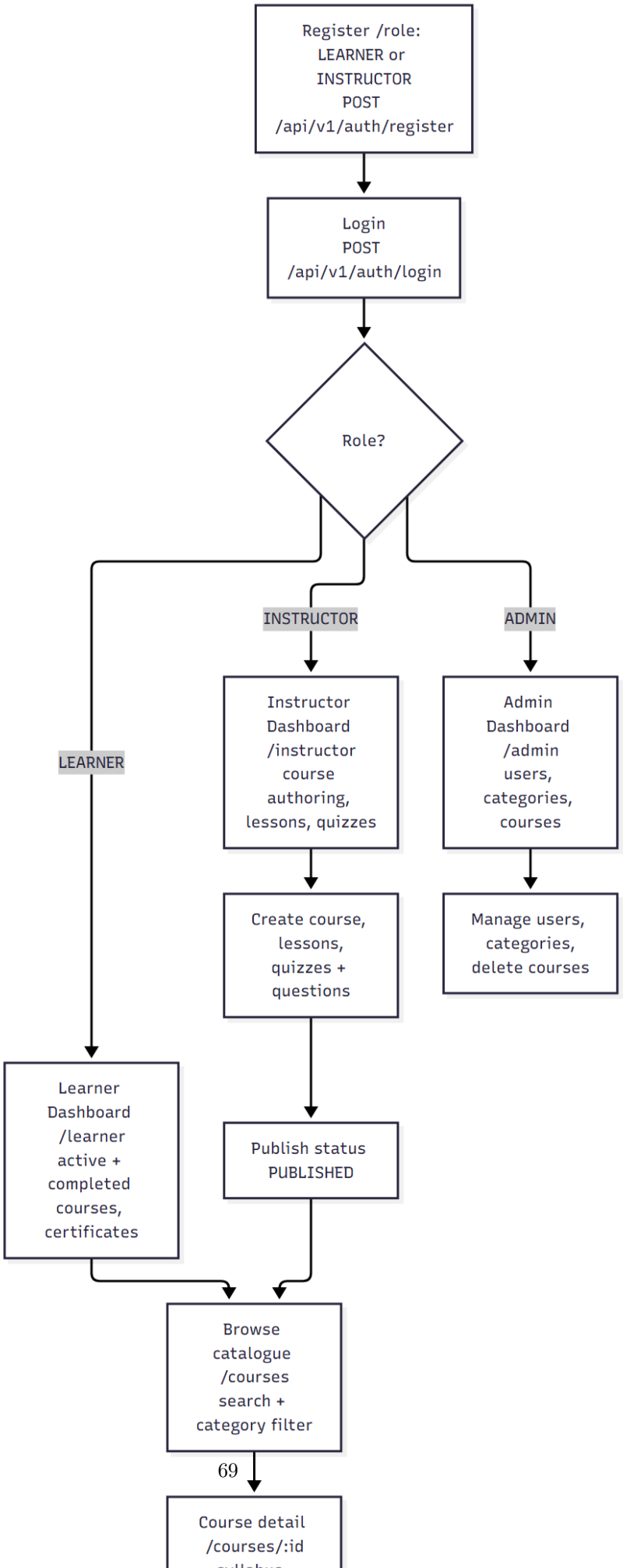
Ownership checks (e.g. instructor may modify only own courses) are performed in service-layer code. Known service-level gaps are recorded as PARTIAL in Section 23.

19 User Flows

19.1 Covered flows

Status: IMPLEMENTED

The end-to-end backend test `backend/src/test/java/com/example/lms/integration/LmsIntegrationTe` exercises the canonical flows: register → login → create course → add lessons → create quiz → enroll → update progress → submit quiz → issue certificate → post discussion and feedback. Frontend routes for each step live in `frontend/src/routes/AppRoutes.jsx` (19 routes including `/courses`, `/courses/:id`, `/courses/:courseId/learn`, `/quizzes/:id`, `/learner`, `/instructor`, `/admin`, `/profile`).


Register /role: LEARNER or INSTRUCTOR POST /api/v1/auth/register
Login POST /api/v1/auth/login
Role?
LEARNER
INSTRUCTOR
ADMIN
Instructor Dashboard /instructor course authoring, lessons, quizzes
Admin Dashboard /admin users, categories, courses
Create course, lessons, quizzes + questions
Manage users, categories, delete courses
Learner Dashboard /learner active + completed courses, certificates
Publish status PUBLISHED
Browse catalogue /courses search + category filter
Course detail /courses/:id

19.2 Flow details

1. **Registration and login:** `/register` → `POST /api/v1/auth/register` → `/login` → `POST /api/v1/auth/login` → JWT stored → role dashboard.
2. **Enrollment:** catalogue (`/courses`) → detail (`/courses/:id`) → enroll (`POST /api/v1/courses/\{courseId`) → learn view unlocked. Duplicate enrollment is rejected with 409.
3. **Learning:** `/courses/:courseId/learn` renders `frontend/src/pages/LearningPage.jsx`; lesson completion posts progress; the dashboard reflects persisted state.
4. **Assessment:** `/quizzes/:id` renders `frontend/src/pages/QuizTakingPage.jsx`; submission is validated and scored server-side; the result view shows the authoritative score.
5. **Certificate:** eligibility is verified by `backend/src/main/java/com/example/lms/service/CertificateSer` the learner downloads the certificate from the learner dashboard.

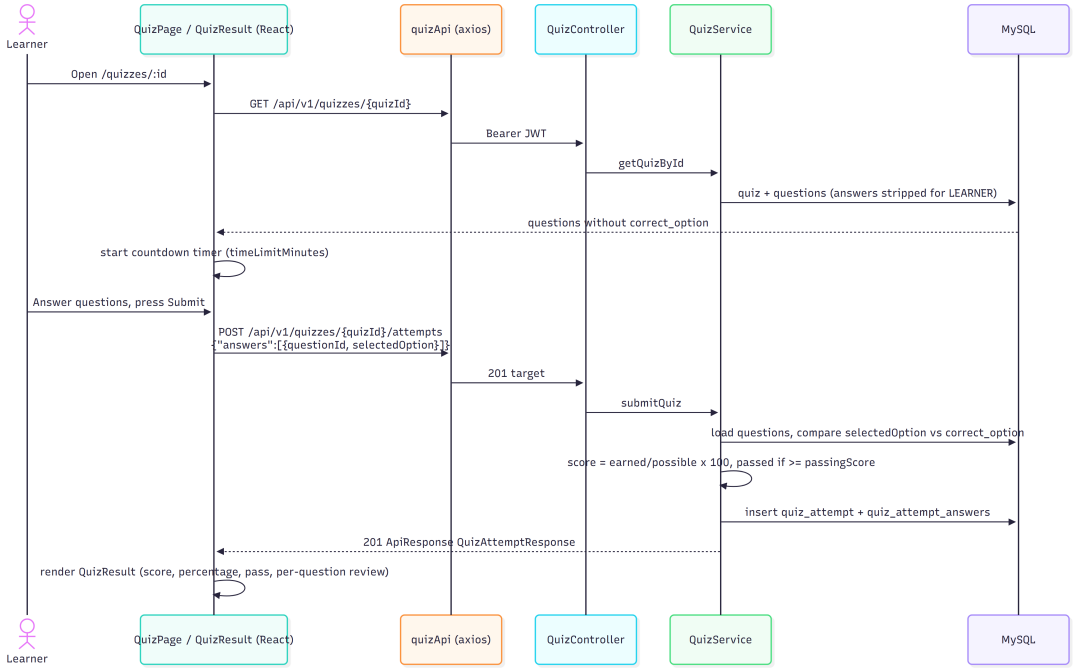


Figure 11: Quiz submission sequence – client attempt, server validation, persisted result

20 Data Flows

20.1 Request lifecycle

Status: IMPLEMENTED

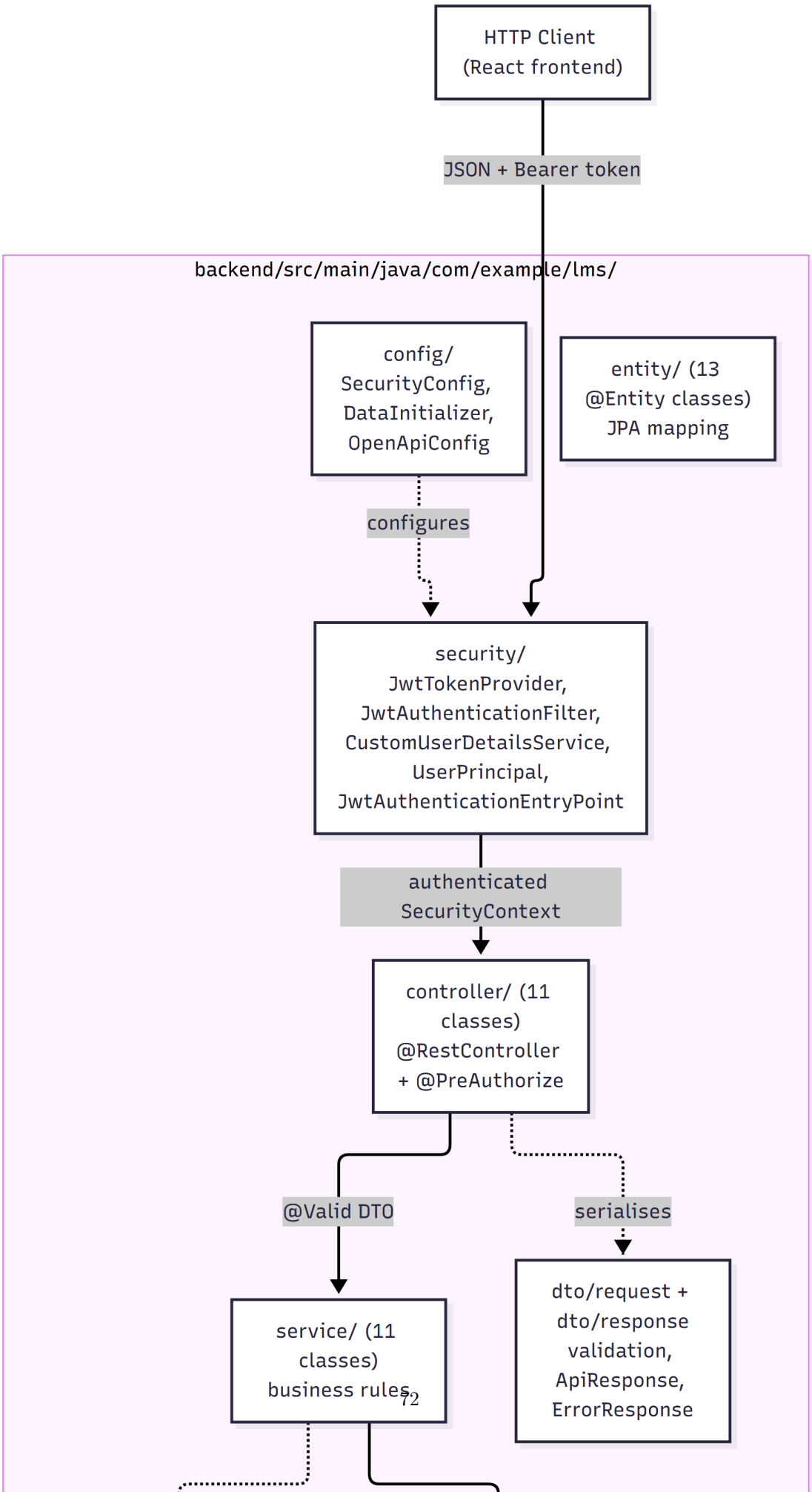
All interactions are synchronous JSON over HTTP: React component → feature API module in `frontend/src/api/` → shared Axios client (`frontend/src/api/client.js`, base URL from `VITE_API_BASE_URL`) → Spring Security filter chain → controller → service (transactional business rules) → JPA repository → MySQL. Responses return through DTOs; the Axios client unwraps envelopes and maps error codes for display.

20.2 Key data chains

- **Progress:** lesson completion event → `POST/PATCH /api/v1/lessons/\{lessonId\}/progress` → `backend/src/main/java/com/example/lms/service/ProgressService.java` → `progress`

table → aggregated `backend/src/main/java/com/example/lms/dto/response/CourseProgressResponse.`

- **Assessment:** answers → `POST /api/v1/quizzes/\{quizId\}/attempts` → `backend/src/main/java/com/` scores server-side → attempt/result rows. The frontend is never authoritative for scores.
- **Certificate:** completion signal → `backend/src/main/java/com/example/lms/service/CertificateServi` verifies enrollment, progress and quiz outcome → `certificates` row → download.


HTTP Client
(React frontend)
JSON + Bearer token
backend/src/main/java/com/example/lms/
config/
SecurityConfig,
DataInitializer,
OpenApiConfig
entity/ (13
@Entity classes)
JPA mapping
configures
security/
JwtTokenProvider,
JwtAuthenticationFilter,
CustomUserDetailsService,
UserPrincipal,
JwtAuthenticationEntryPoint
authenticated
SecurityContext
controller/ (11
classes)
@RestController
+ @PreAuthorize
@Valid DTO
serialises
service/ (11
classes)
business rules
dto/request +
dto/response
validation,
ApiResponse,
ErrorResponse

20.3 What does not exist

Status: IMPLEMENTED (absence verified)

No event bus, message queue, cache layer, search index, object store, or background worker. No optimistic offline sync: the backend database is the single source of truth.

21 UI/UX

21.1 Page inventory

Status: IMPLEMENTED

Public: `/` (`frontend/src/pages/HomePage.jsx`), `/login`, `/register`, `/courses` (`frontend/src/pages/Cou`
`/courses/:id` (`frontend/src/pages/CourseDetailPage.jsx`), `/practice` and `/courses/:courseId/practic`
Authenticated: `/courses/:courseId/learn` (`frontend/src/pages/LearningPage.jsx`), `/quizzes/:id`
(`frontend/src/pages/QuizTakingPage.jsx`), role dashboards (`frontend/src/components/LearnerDashboar`
`frontend/src/components/InstructorDashboard.jsx`, `frontend/src/components/AdminDashboard.jsx`),
`/profile`, plus `*` fallback (`frontend/src/pages/NotFoundPage.jsx`).

21.2 State contract

Status: IMPLEMENTED

Per CONTRACT.md section 16 every major page provides loading, success, empty, error and retry states; the frontend must never silently render empty data on API failure. Shared chrome lives in `frontend/src/components/Navbar.jsx` and `frontend/src/components/Footer.jsx`; reusable primitives live in `frontend/src/components/ui/` and `frontend/src/components/common/`.

21.3 Responsive and accessible behaviour

Status: IMPLEMENTED (with gaps)

Tailwind utilities plus Ant Design components adapt desktop/tablet/mobile; cards must not overflow and forms must remain usable per the contract. **Status:** RECOMMENDED: a systematic accessibility pass (focus order, labels, contrast audit against WCAG 2.1) is not evidenced in the repository.

22 Design System

22.1 Tokens

Status: IMPLEMENTED

The Ant Design theme in `frontend/src/App.jsx` sets `colorPrimary: \#000000`, 4px radius and 40px control height with component overrides; Tailwind tokens live in `frontend/tailwind.config.js`. Brand accents observed in code: gradient `linear-gradient(90deg, \#fc4c02, \#ef2cc1, \#bdbbff)` (`frontend/src/components/ui/GradientRibbon.jsx`), mint `\#c8f6f9`, dark canvas `\#010120`. Product brand is Edvanta (`frontend/index.html`, `frontend/src/components/Footer.jsx`).

22.2 Component library

Status: IMPLEMENTED

Ant Design 5.22.2 provides tables, modals, forms, tabs and message toasts; project primitives in `frontend/src/components/ui/` (including `GradientRibbon.jsx`) and `frontend/src/components/common/`

wrap them. Icons come from `@ant-design/icons` 5.5.2.

22.3 Usage rule

Status: RECOMMENDED

New screens should reuse the App-level theme and Tailwind tokens rather than hardcoding colours or spacing, keeping the black-first Edvanta identity consistent.

23 Security

23.1 Implemented controls

Status: IMPLEMENTED

BCrypt password hashing, stateless JWT authentication with expiry (`backend/src/main/java/com/example`, Bean Validation on DTOs, server-side role enforcement, consistent error envelopes that never leak stack traces or credentials, and JPA parameter binding (no string-concatenated SQL). Secrets are externalised through `.env.example` (`DB_URL`, `DB_USERNAME`, `DB_PASSWORD`, `JWT_SECRET`, `JWT_EXPIRATION`).

23.2 Findings

Status: IMPLEMENTED (gaps recorded, not invented)

Table 52: Security findings grounded in repository evidence

ID	Finding	Evidence
S-01	JWT in `localStorage`; no revocation or rotation	`frontend/src/context/AuthContext.jsx`; no refresh/token-blocklist code
S-02	`DataInitializer` seeds demo users with hardcoded BCrypt passwords and no profile guard	`backend/src/main/java/com/example/lms/config/Dat`
S-03	Example `JWT_SECRET` committed in `.env.example`; must be rotated in production	File contents; no vault integration
S-04	No rate limiting, lockout, password recovery, or email verification	Absence verified in controllers and config
S-05	`ddl-auto: update` in configuration; schema history untracked	Backend application config

23.3 Production requirements

Status: RECOMMENDED

HTTPS, rotated secrets via a manager, CORS allow-list review, rate limiting on auth endpoints, security headers, audit logging, and removal or guarding of seed data before release. See OWASP Top Ten [19] and the JWT cheat sheet [20].

24 Performance

24.1 Observed characteristics

Status: INFERRED

The application is a synchronous monolith with no cache, CDN, pagination contract, or query hints evidenced in the repository. Every list view is a direct JPA query; every interaction is a full HTTP round trip. No load-test results, budgets, or APM wiring exist in the repository (**Status:** UNKNOWN for all numeric targets).

24.2 Bottleneck risks

Status: INFERRED

Unbounded course/lesson/discussion listings, N+1-prone entity graphs, and server-side scoring loops are the most likely hotspots. The Vite production bundle (`npm run build` into `frontend/dist/`) is the only evidenced optimisation.

24.3 Recommendations

Status: RECOMMENDED

Add pagination to list endpoints, index-verify slow queries against MySQL 8 [16], introduce response caching only after measurement, set performance budgets (e.g. catalogue p95), and instrument before scaling claims are made.

25 Scalability

25.1 Current posture

Status: INFERRED

The stateless JWT design allows horizontal scaling of the Spring Boot tier, but the single MySQL instance, file-less local state, and absence of caching, read replicas, or async processing bound vertical growth. No horizontal-scaling configuration (compose file, replica policy, migration runner) exists in the repository.

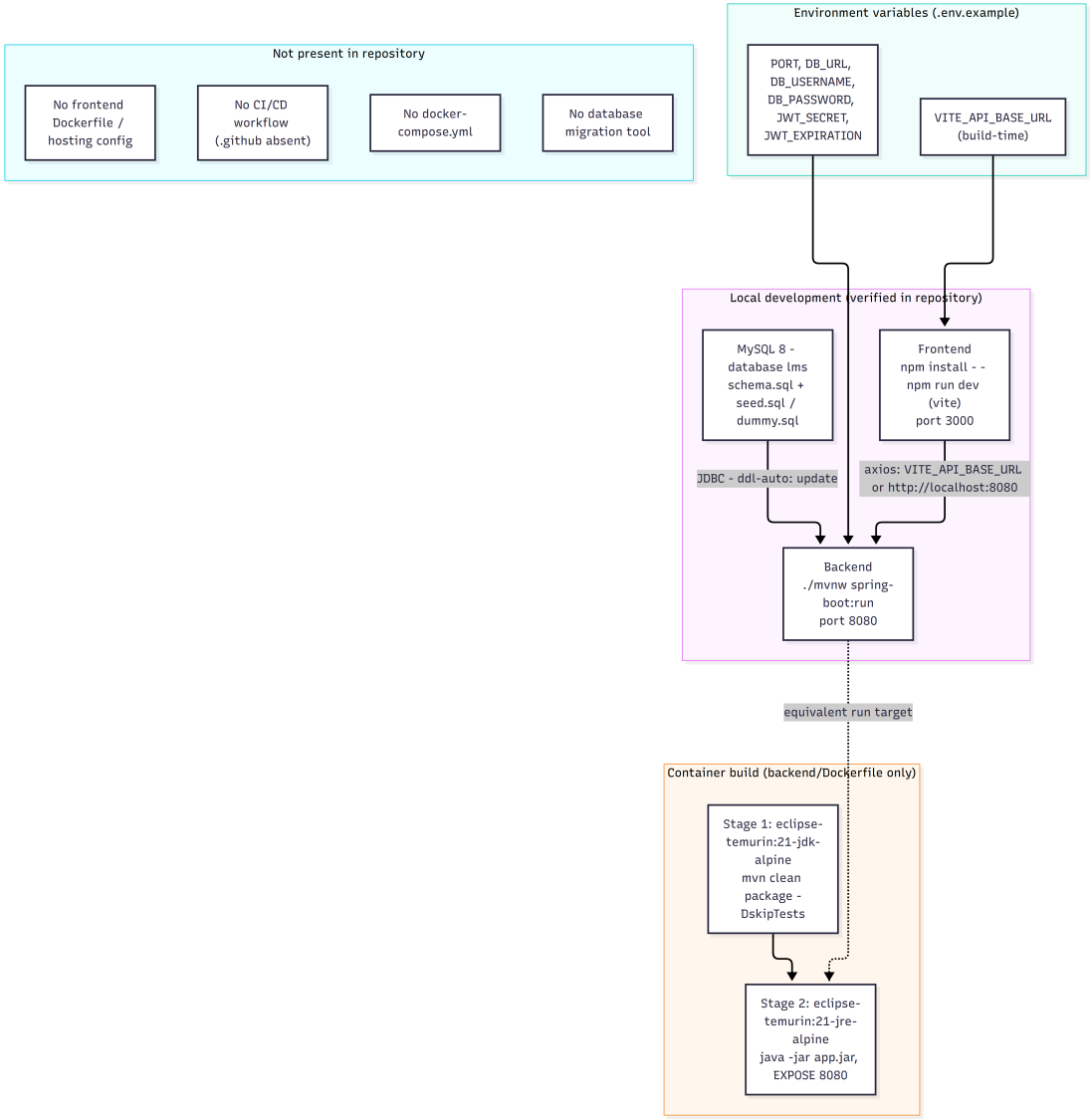


Figure 13: Deployment view – the only containerised component is the backend JAR

25.2 Scaling path

Status: RECOMMENDED

Containerise the frontend static bundle, add a compose or orchestrator topology, externalise sessions only if introduced, adopt versioned migrations before multi-instance deploys, then consider read replicas, caching, and CDN for static assets. Do not claim elastic scalability until the database and migration story are hardened.

26 Testing

26.1 Backend tests

Status: IMPLEMENTED

22 test methods across 10 classes under `backend/src/test/`, including the end-to-end `backend/src/test/java/com/example/lms/integration/LmsIntegrationTest.java` (register, course, lessons, quiz, enroll, progress, certificate, discussion, feedback) with H2 as the test database.

Run with `./mvnw test` from `backend/`.

26.2 Frontend tests

Status: IMPLEMENTED

39 test methods across 8 files under `frontend/src/test/` using Vitest 2.1.8 with Testing Library and jsdom. Run with `npm test` (i.e. `vitest run`) from `frontend/`.

26.3 Coverage contract

Status: IMPLEMENTED (per CONTRACT.md sections 18–19)

Backend must cover registration, login, invalid login, JWT validation, role authorization, course CRUD, enrollment including duplicates, progress, quiz submission and scoring, discussions, feedback, certificates, and invalid/unauthorized/forbidden/validation cases. Frontend must cover auth forms, route protection, catalogue, details, enrollment, lesson navigation, progress display, quiz interaction, results, discussions, feedback, certificates, dashboards, and loading/empty/error/responsive states. Five integration scenarios (register/login, enrollment, progress, assessment, certificate) must pass end to end.

27 Deployment

27.1 Verified development topology

Status: IMPLEMENTED

MySQL 8 on `localhost:3306` (schema from `database/schema.sql`) plus `./mvnw spring-boot:run` on :8080 plus Vite on :3000 with an `/api` proxy (`frontend/vite.config.js`). Backend base URL is configured once via `VITE_API_BASE_URL` (`.env.example`) and consumed through `frontend/src/api/client.j`

27.2 Containerisation

Status: PARTIAL

`backend/Dockerfile` is a multi-stage build (Maven on JDK 21 alpine, runtime on JRE 21 alpine, `EXPOSE 8080`). There is no frontend Dockerfile and no `docker-compose.yml`; production topology, TLS termination, backups, log shipping and health-check wiring are undefined.

27.3 Production checklist

Status: RECOMMENDED

HTTPS, rotated secrets, managed MySQL with backups, versioned migrations, CORS allowlist, production JWT secret, structured logging, monitoring, and a defined rollback procedure. See also Sections 28 and 30.

28 CI/CD

28.1 Current state

Status: IMPLEMENTED (absence verified)

No `.github/` workflows, no pipeline configuration, and no automated gates exist in the repository. Builds and tests are manual: `./mvnw test` and `npm test` / `npm run build`.

28.2 Recommended pipeline

Status: RECOMMENDED (not implemented)

On pull request: backend build plus tests, frontend install plus lint/typecheck plus tests plus production build, and a docs-link check. On merge to `main`: image build, migration dry-run, and staged deploy. Branch protections should enforce the CONTRACT.md review rules for shared files. No pipeline must be presented as existing.

29 Integrations

29.1 Embedded practice editor

Status: IMPLEMENTED

Course-attached coding practice embeds OneCompiler (`frontend/src/pages/practice/PracticePage.jsx`, `frontend/src/pages/practice/CoursePracticePage.jsx`, data in `frontend/src/data/`) via iframe with `postMessage` starter-code injection [22]. It is an isolated frontend dataset with no backend dependency by explicit design.

29.2 External origins

Status: IMPLEMENTED

YouTube (lesson video references), Google Fonts, and Unsplash imagery are loaded as third-party origins outside the trust boundary and receive no user secrets.

29.3 Not integrated

Status: NOT APPLICABLE

No payment gateway, email provider, SSO, analytics, or object storage is wired in v1.0 (explicitly excluded by CONTRACT.md section 5.2).

30 Configuration

30.1 Backend configuration

Status: IMPLEMENTED

Centralised Spring configuration (`application.yml` or equivalent) with sensitive values loaded from environment: `DB_URL`, `DB_USERNAME`, `DB_PASSWORD`, `JWT_SECRET`, `JWT_EXPIRATION`, `PORT` – all documented in `.env.example`. Never commit real credentials; production must inject them through the environment or a secret manager.

30.2 Frontend configuration

Status: IMPLEMENTED

The backend base URL is configured once as `VITE_API_BASE_URL` and referenced by every module in `frontend/src/api/` through the shared client. No component may hardcode `http://localhost:8080/api/...` (CONTRACT.md section 21).

30.3 Database bootstrap

Status: IMPLEMENTED

`database/schema.sql` (13 tables), `database/seed.sql`, and `database/dummy.sql` define the hand-written schema; the runtime uses Hibernate `ddl-auto: update`. The drift risk between the two is recorded in Section 35.

31 SEO

31.1 Current state

Status: PARTIAL

The SPA shell (`frontend/index.html`, Vite build into `frontend/dist/`) carries literal Edvanta metadata supporting "LMS", "structured courses", "assessments" and "certificates". Client-side rendering means crawlers see a thin shell; no sitemap, robots policy, canonical URLs, structured data, or prerendering is evidenced.

31.2 Recommendations

Status: RECOMMENDED

Add unique titles/descriptions per public route (`/`, `/courses`, `/courses/:id`), semantic HTML, canonical links, a sitemap, and server-side rendering or prerendering for the catalogue if organic acquisition matters. Do not claim search visibility that has not been measured.

32 AEO (Answer-Engine Optimisation)

32.1 Current state

Status: PARTIAL

Course titles, descriptions and lesson text in MySQL are quotable answer sources, but no Q&A schema, FAQ blocks, or extractable how-to summaries are provided for answer engines.

32.2 Recommendations

Status: RECOMMENDED

Expose concise question/answer summaries per course (prerequisites, outcomes, duration), add FAQ structured data on public course pages, and keep factual claims (durations, levels, certificates) consistent between the UI and the API so assistants quote correct information.

33 GEO (Generative-Engine Optimisation)

33.1 Current state

Status: PARTIAL

The same course/lesson corpus that serves learners is the only corpus generative engines could cite; no `llms.txt`, AI-use policy, or structured catalogue feed exists.

33.2 Recommendations

Status: RECOMMENDED

Publish a machine-readable course catalogue summary, keep public descriptions factual and up to date, and define which content may be summarised or quoted by third-party assistants. No

GEO capability is implemented until these artefacts exist.

34 Known Issues

34.1 Recorded defects and inconsistencies

Status: IMPLEMENTED (observations, not inventions)

Table 53: Known issues with repository evidence

ID	Issue	Evidence / impact
K-01	`CertificateService` writes grade `DISTINCTION` while the column default is `PASS`	Cosmetic inconsistency in `certificates` rows
K-02	Quiz percentage ignores unanswered questions (denominator only counts submitted questions)	Behaviour in quiz scoring service; inflates partial attempts
K-03	README roadmap lists discussions/feedback as future work although both are implemented	`README.md` Phase 2 vs. discussion/feedback controllers
K-04	Seed users with hardcoded passwords run on any empty database (no profile guard)	`backend/src/main/java/com/example/lms/config/`
K-05	Hand-written `database/schema.sql` can drift from JPA entities under `ddl-auto: update`	No migration tooling; untested in CI

Each item above is observable in the checked-in sources. Fixes belong on the roadmap (Section 36) and must update the affected docs.

35 Technical Debt

35.1 Debt register

Status: INFERRED (each item traces to repository evidence)

Table 54: Technical debt register

ID	Debt	Consequence / remedy
TD-01	No versioned migrations; `ddl-auto: update`	Schema history untracked; adopt Flyway/Liquibase
TD-02	No CI/CD pipeline	Regressions undetected; add build/test gates
TD-03	Seed data unguarded by profile	Demo accounts in prod risk; add profile guard
TD-04	JWT without revocation/rotation	Cannot revoke sessions; add refresh + blocklist
TD-05	Unbounded list endpoints | N+1 risk	Catalogue scale risk; add pagination, fetch plans
TD-06	Frontend/backend contract only in prose	Drift risk; add OpenAPI diff check in CI
TD-07	README roadmap drifted from code | missing LICENSE | no compose file	Onboarding friction; reconcile docs, add licence

36 Roadmap

36.1 Reconciled position

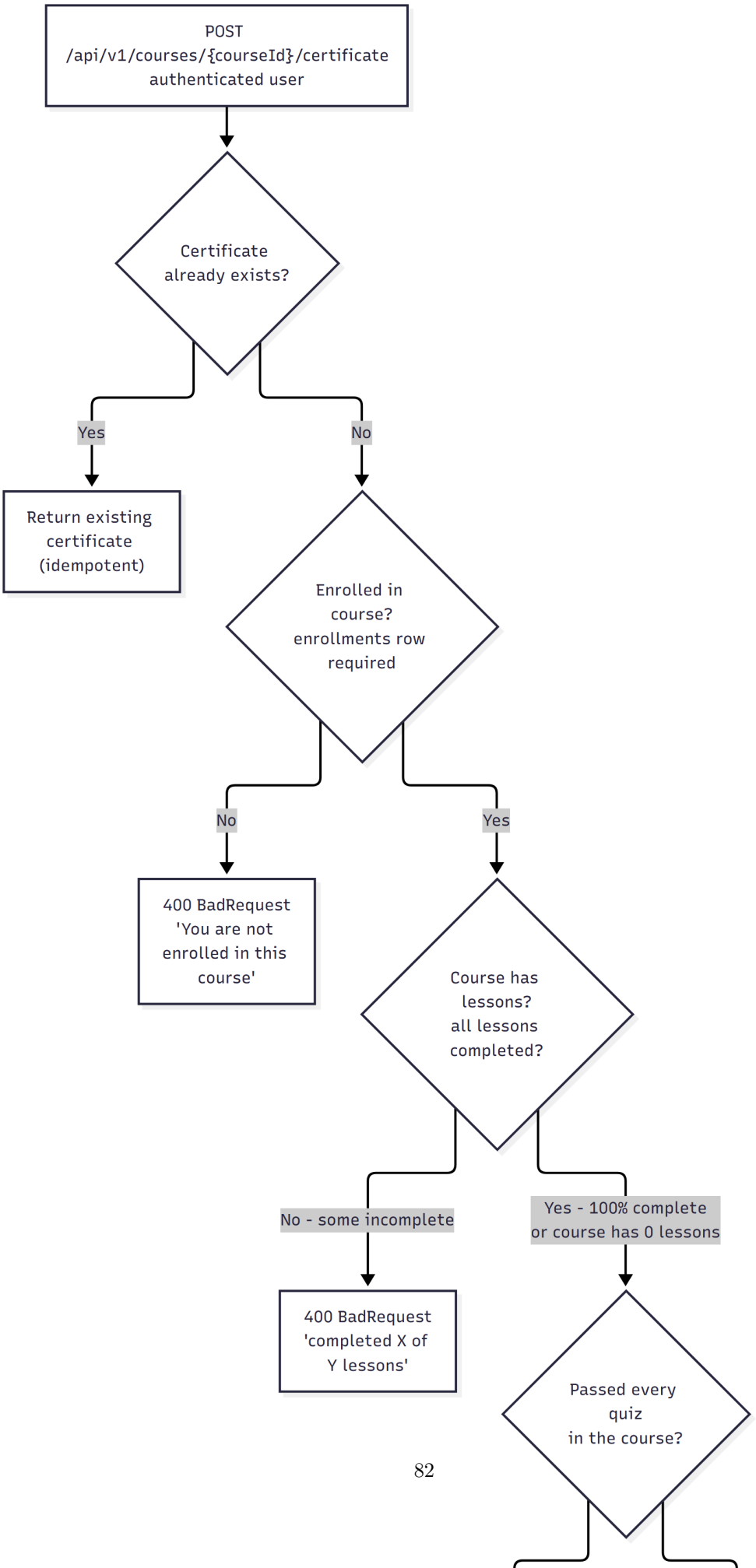
Status: INFERRED

`README.md` marks Phase 1 (core LMS) complete and Phases 2–4 unchecked, yet discussions and feedback are already implemented – the README has drifted (K-03). The reconciled position: the core learning lifecycle is implemented and covered by an integration test; engagement, instructor-platform and advanced-platform items remain a mix of present and absent.

36.2 Phased plan

Status: RECOMMENDED (not committed scope)

Next (hardening): fix K-01/K-02 scoring and grade consistency, guard seed data, adopt versioned migrations, add CI gates, rotate secrets, reconcile README. **Engagement:** notifications, learning analytics, improved learner dashboard. **Instructor platform:** authoring UX, lesson management, performance analytics. **Advanced:** certificate verification, search/filtering, recommendations, password recovery, monitoring, scalable deployment. CONTRACT.md section 5.2 exclusions (payments, live classes, mobile, AI tutoring, SSO, multi-tenancy) require formal agreement before work begins.


POST
/api/v1/courses/{courseId}/certificate
authenticated user
Certificate
already exists?
Yes
No
Return existing
certificate
(idempotent)
Enrolled in
course?
enrollments row
required
No
Yes
400 BadRequest
'You are not
enrolled in this
course'
Course has
lessons?
all lessons
completed?
No - some incomplete
Yes - 100% complete
or course has 0 lessons
400 BadRequest
'completed X of
Y lessons'
Passed every
quiz
in the course?

37 Development Guidelines

37.1 Layer discipline

Status: IMPLEMENTED (per CONTRACT.md)

Backend: controller (HTTP + validation) → service (business rules, transactions) → repository (persistence) → database; entities in `backend/src/main/java/com/example/lms/entity/`, contracts in `backend/src/main/java/com/example/lms/dto/`. Never place queries in controllers or business rules in DTOs. Frontend: pages in `frontend/src/pages/`, reusable UI in `frontend/src/components/`, backend communication only through `frontend/src/api/`, shared logic in `frontend/src/hooks/`, shared state in `frontend/src/context/`.

37.2 Working rules

Status: IMPLEMENTED

Respect workstream ownership (`frontend/**` vs. `backend/**` + `database/**`); API and schema changes require agreement before integration; keep commits focused (`feat(frontend): ...`, `fix(backend): ...`); run `./mvnw test` and `npm test` before every PR; never commit secrets.

38 Contribution Guide

38.1 Workflow

Status: IMPLEMENTED (per CONTRACT.md sections 22–25)

Develop on workstream branches (`frontend/...`, `backend/...`), never directly on `main`. Every PR states summary, changes, files, tests run, expected behaviour and known issues. Shared files (`README.md`, `CONTRACT.md`, `.env.example`, API and schema contracts, auth rules, major dependencies) require cross-workstream review. AI-assisted contributors must read `CONTRACT.md` and `README.md` first, stay inside the assigned workstream, follow canonical naming, and report failures honestly.

38.2 Definition of Done (abridged)

Status: IMPLEMENTED

Feature, auth, course, enrollment, learning, assessment, discussion/feedback, certificate, frontend, backend and integration checklists in CONTRACT.md section 32 must all pass – including real-backend integration with no required mock data and verified end-to-end flows.

39 Glossary

Table 55: Project glossary – terms as used in this repository

Term	Meaning
Edvanta	Product UI brand (title in `frontend/index.html`, footer in `frontend/src/components/Footer.jsx`)
Learner / Instructor / Admin	The three `users.role` values enforced by Spring Security and route guards
Course / Lesson	A course (`database/schema.sql courses`) owns ordered lessons (`lessons`, `order_index`)
Enrollment	Link between a learner and a course (`enrollments`, unique per user+course)
Progress	Per-lesson completion state aggregated to course percentage
Quiz / Question / Attempt / Result	Assessment chain: quiz owns questions; a submission creates an attempt scored server-side into a result
Discussion / Feedback	Course-scoped threads/replies and learner ratings
Certificate	Completion credential issued after server-side eligibility verification
DTO	Data-transfer object in `backend/src/main/java/com/example/lms/dto/` defining the API contract
JWT	Stateless signed token (JJWT 0.12.6) authorising API calls
springdoc	OpenAPI/Swagger generator serving `/v3/api-docs` and Swagger UI
Vite	Frontend build/dev server (port 3000, proxies `/api`)

A References

The following external standards, specifications and official documentation were consulted for the technologies actually used in this repository. No fictitious citations are included.

References

[1] Spring Documentation. *Spring Boot Reference Documentation* (3.3.x). https://docs.spring.io/spring-boot/reference/

[2] Spring Security Documentation. *Spring Security Reference*. https://docs.spring.io/spring-security/reference/

[3] Spring Data JPA Documentation. https://docs.spring.io/spring-data/jpa/reference/

[4] JJWT (JSON Web Token for Java). *JJWT 0.12.x Documentation*. https://github.com/jwtk/jjwt

[5] IETF RFC 7519. *JSON Web Token (JWT)*. https://datatracker.ietf.org/doc/html/rfc7519

[6] IETF RFC 7517. *JSON Web Key (JWK)*. https://datatracker.ietf.org/doc/html/rfc7517

[7] springdoc-openapi. *OpenAPI 3 for Spring Boot* (2.6.0). https://springdoc.org/

[8] OpenAPI Initiative. *OpenAPI Specification 3.1*. https://spec.openapis.org/oas/latest.html

[9] React Documentation. *React 18 – Getting Started*. https://react.dev/

[10] React Router Documentation (v6). https://reactrouter.com/

[11] Vite Documentation. https://vite.dev/

[12] Vitest Documentation. https://vitest.dev/

[13] Ant Design 5 Documentation. https://ant.design/docs/react/introduce

[14] Tailwind CSS Documentation (v3). https://tailwindcss.com/docs

[15] Axios Documentation. https://axios-http.com/

[16] MySQL Reference Manual (8.0). https://dev.mysql.com/doc/refman/8.0/en/

[17] Hibernate ORM Documentation. https://docs.jboss.org/hibernate/orm/current/reference/en/html_single/

[18] *The Bcrypt Password Hashing Function*, RFC 7914. https://datatracker.ietf.org/doc/html/rfc7914

[19] OWASP Foundation. *OWASP Top Ten*. https://owasp.org/www-project-top-ten/

[20] OWASP Foundation. *JSON Web Token Cheat Sheet for Java*. https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

[21] WebAIM. *Web Content Accessibility Guidelines (WCAG) 2.1 Overview*. https://www.w3.org/WAI/standards-guidelines/wcag/

[22] OneCompiler. *Embed Code Editor*. https://onecompiler.com/embed

[23] Pandoc. *Pandoc User's Manual* (for optional DOCX conversion). https://pandoc.org/MANUAL.html

[24] Mermaid Live Editor / mermaid-cli. https://mermaid.js.org/